ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

4IASDT G1

Rapport de Projet

Système Distribué de Gestion des Incidents IT

Architecture Microservices, Intégration et Traçabilité

Réalisé par :

Bilal EL-Haoudar - Ilyas Oulmidi - Ahmed Nahri - Ilyas Choqri - Anas Lhamri

Année Universitaire : 2025 / 2026

Table des matières

Introduction générale **7**

1 Microservice Déclaration des Incidents **9**

1.1 Introduction 9

1.2 Rôle dans le système distribué 9

1.3 Architecture technique 10

1.3.1 Package model 10

1.3.2 Package repository 10

1.3.3 Package service 11

1.3.4 Package controller 11

1.3.5 Package dto 11

1.3.6 Package client 11

1.4 Base de données 11

1.5 API REST exposées 12

1.6 Fonctionnalités spécifiques 12

1.6.1 Mode dégradé (Résilience) 12

1.6.2 Interface web intégrée 13

1.6.3 Documentation Swagger 13

1.7 Diagramme de séquence 13

1.8 Tests réalisés 14

1.9 Conclusion 14

2 Microservice Affectation & Traitement **15**

2.1 Introduction 15
2.2 Rôle dans le système distribué 15
2.3 Architecture technique 16
2.3.1 Package model 16
2.3.2 Package repository 16
2.3.3 Package service 16
2.3.4 Package controller 16
2.3.5 Package config 17
2.4 Base de données 17
2.5 API REST exposées 17
2.6 Interface Web intégrée 18
2.7 Diagramme de séquence 18
2.8 Tests réalisés 19
2.9 Conclusion 20

3 Microservice Utilisateurs et Équipes 21

3.1 Introduction 21
3.2 Rôle dans le système distribué 21
3.3 Architecture technique 22
3.3.1 Package model 22
3.3.2 Package repository 22
3.3.3 Package service 22
3.3.4 Package controller 23
3.3.5 Package exception 23
3.3.6 Package config 23
3.4 Base de données 23
3.5 API REST exposées 24
3.6 Fonctionnalités bonus 25
3.6.1 Auto-stabilisation 25
3.6.2 Préparation cloud 26

3.6.3 Interface web 26
3.6.4 Documentation Swagger 27
3.7 Diagramme de séquence 27
3.8 Tests réalisés 28
3.9 Conclusion 29

4 Microservice Équipements et Ressources IT 30
4.1 Introduction 30
4.2 Rôle dans le système distribué 30
4.3 Architecture technique 31
4.3.1 Package model 31
4.3.2 Package repository 31
4.3.3 Package service 31
4.3.4 Package controller 31
4.3.5 Packages exception et config 31
4.4 Base de données 32
4.5 API REST exposées 32
4.6 Fonctionnalités bonus 32
4.6.1 Tableau de bord centralisé (Interface Web React) 32
4.6.2 Préparation Cloud et Auto-stabilisation 33
4.6.3 Documentation Swagger 33
4.7 Conception et Diagrammes 34
4.7.1 Diagramme de classes 34
4.7.2 Diagramme de séquence 34
4.8 Tests réalisés 35
4.9 Conclusion 35

5 Microservice Notifications et Historique 36
5.1 Introduction 36
5.2 Rôle dans le système distribué 36

5.3 Architecture technique 37
5.3.1 Package model 37
5.3.2 Package dto 37
5.3.3 Package repository 37
5.3.4 Package service 38
5.3.5 Package controller 38
5.3.6 Package config 38
5.4 Base de données 38
5.5 API REST exposées 39
5.6 Diagrammes de séquence 40
5.7 Tests réalisés 42
5.8 Conclusion 43

6 Schéma des microservices et description de larchitecture globale 44
6.1 Vue densemble 44
6.2 Ordre logique du flux métier 46
6.3 Description de larchitecture globale 46
6.4 Convention de ports retenue pour lintégration globale 47

7 Répartition claire du travail 48
7.1 Principe de répartition 48
7.2 Répartition des microservices 48
7.3 Responsabilités transversales 49
7.4 Synthèse 49

8 Réalisation de linterface finale 50
8.1 Introduction 50
8.2 Vue densemble du système 50
8.3 Module de déclaration des incidents 51
8.4 Module daffectation et de traitement 52
8.5 Module de gestion des équipements 53

8.6 Module de gestion des utilisateurs 54
8.7 Module de journalisation et dactivité 55
8.8 Module de configuration centralisée 56
8.9 Apport fonctionnel de linterface finale 57
8.10 Synthèse 57

9 Conclusion générale **59**

Table des figures

1.1 Figure 1.1 – Interface web intégrée du Microservice 1 (Déclaration) 13

1.2 Figure 1.2 – Diagramme de séquence du Microservice 1 (Déclaration) . . . 14

2.1 Figure 1 – Interface Web du Microservice Affectation & Traitement 18

2.2 Figure 2 – Diagramme de séquence du Microservice Affectation 19

3.1 Figure 1.1 – Interface web du microservice Utilisateurs et Équipes 27

3.2 Figure 1.2 – Diagramme de séquence du traitement dun incident dans le système distribué 28

4.1 Figure 1.1 – Dashboard React : Supervision globale du système distribué . 33

4.2 Figure 1.2 – Diagramme de classes du microservice Équipements 34

4.3 Figure 1.3 – Diagramme de séquence : Signalement dune panne 35

5.1 Figure 1 – Séquence : création dincident vers notification & historique . . . 41

5.2 Figure 2 – Séquence : ajout direct dune entrée historique depuis le microservice Équipements 42

6.1 Schéma global des microservices du projet 45

8.1 Figure – Interface finale : vue densemble du système distribué 51

8.2 Figure – Interface finale : module de déclaration des incidents 52

8.3 Figure – Interface finale : module daffectation et de traitement 53

8.4 Figure – Interface finale : module de gestion des équipements 54

8.5 Figure – Interface finale : module de gestion des utilisateurs 55

8.6 Figure – Interface finale : module de logs et activité 56

8.7 Figure – Interface finale : module de configuration des microservices 57

Introduction générale

Dans le cadre du module Systèmes distribués, notre groupe a réalisé un système distribué de gestion des incidents IT fondé sur une architecture microservices. L'objectif global du projet est de modéliser un cycle de traitement complet des incidents informatiques, depuis leur déclaration jusqu'à leur affectation, leur suivi, leur historisation et la consultation de leur contexte technique. Le système repose sur le principe de séparation des responsabilités : chaque microservice possède sa propre base de données, sa propre logique métier et ses propres API REST, tout en collaborant avec les autres services via des appels HTTP.

Cette architecture permet de représenter un fonctionnement réaliste. Lorsqu'un incident est déclaré, les informations ne sont pas validées dans une seule application monolithique ; elles sont vérifiées auprès de plusieurs services spécialisés. Le microservice Déclaration des incidents joue le rôle de point d'entrée. Il vérifie l'existence du demandeur via le microservice Utilisateurs et Équipes, valide l'équipement concerné via le microservice Équipements et Ressources IT, puis enregistre le ticket et notifie le microservice Notifications et Historique. Ensuite, le microservice Affectation et Traitement prend le relais pour l'assignation à un technicien, la mise à jour des statuts et la clôture. Enfin, le microservice Notifications et Historique conserve la mémoire transversale du système, tandis que le microservice Utilisateurs et Équipes fournit les identités et les équipes techniques nécessaires aux autres opérations.

Le présent document a pour objectif de regrouper l'ensemble des rapports individuels réalisés par les membres du groupe dans un rapport unique cohérent. Une introduction générale détaillée est placée en ouverture, suivie des chapitres de chaque microservice, conservés dans l'ordre logique du sujet traité : Déclaration des incidents, Affectation & Traitement, Utilisateurs & Équipes, Équipements & Ressources IT, Notifications & Historique. Les figures et captures d'écran originales ont été remplacées par des emplacements à compléter afin de faciliter l'intégration finale sur Overleaf.

Au-delà de la simple juxtaposition des rapports individuels, ce document ajoute également deux chapitres de synthèse : un chapitre consacré au schéma global des microservices et à la description de l'architecture distribuée, puis un chapitre de répartition du travail explicitant clairement la contribution de chaque membre. L'ensemble fournit ainsi une

vision complète du projet, à la fois technique, fonctionnelle et organisationnelle.

Chapitre 1

Microservice Déclaration des Incidents

1.1 Introduction

Dans le cadre du projet de système distribué de gestion des incidents IT, chaque étudiant a développé un microservice distinct. Ma contribution porte sur le microservice Déclaration des Incidents, qui constitue le point dentrée du système et la porte daccès au cycle de traitement des incidents. Ce microservice fonctionne sur le port 8081 et est développé avec Java 21, Spring Boot 3.1.5 et MySQL via XAMPP.

Contrairement aux autres microservices qui gèrent des ressources statiques (utilisateurs, équipements), ce microservice gère des événements dynamiques (les incidents) et orchestre les validations initiales en interrogeant les services de référence avant toute persistance.

1.2 Rôle dans le système distribué

Le microservice Déclaration des Incidents joue le rôle de point dentrée (*entry point*) dans larchitecture globale. Aucun incident ne peut entrer dans le système sans passer par ce service. Il est responsable de la création, de la consultation et du filtrage des incidents, tout en assurant la cohérence des données via des validations externes.

Les interactions avec les autres microservices sont les suivantes :

— Le microservice Déclaration appelle MS3 (Utilisateurs et Équipes, port 8083) via `GET /api/utilisateurs/{id}/existe` pour vérifier que le demandeur existe avant denregistrer lincident. Si le demandeur est invalide, lincident est refusé.

— Le microservice Déclaration appelle MS4 (Équipements et Ressources IT, port

8084) via `GET /api/equipements/{id}/existe` pour valider léquipement concerné par lincident.

— Le microservice Déclaration appelle MS5 (Notifications et Historique, port 8085) via `POST /api/notifications/evenement` pour signaler la création dun nouvel incident et permettre lenregistrement dans lhistorique global.

— Le microservice Affectation et Traitement (MS2, port 8082) interroge ce microservice via `GET /api/incidents/{id}` pour récupérer les informations dun incident avant de laffecter à un technicien, et via `GET /api/incidents/{id}/existe` pour vérifier son existence.

— Le microservice Reporting (MS6, port 8086) appelle `GET /api/incidents` pour récupérer la liste complète des incidents afin de générer des statistiques et tableaux de bord.

1.3 Architecture technique

Notre microservice suit larchitecture en couches standard de Spring Boot, organisée en sept packages principaux pour garantir la séparation des responsabilités.

1.3.1 Package model

Le package `model` contient la classe entité principale du système. La classe `Incident` est annotée avec `@Entity` et correspond à la table `incidents` dans la base de données MySQL. Elle contient les attributs suivants : identifiant auto-généré, titre, description, date de déclaration (automatique), priorité (enum : FAIBLE, MOYENNE, HAUTE, CRITIQUE), catégorie (enum : RESEAU, MATERIEL, LOGICIEL, SECURITE, AUTRE), statut (enum : OUVERT, EN_COURS, RESOLU, CLOTURE), identifiants et noms du demandeur et de léquipement concerné.

1.3.2 Package repository

Le package `repository` contient linterface `IncidentRepository` qui étend `JpaRepository<Incident, Long>`. Cette interface hérite automatiquement des méthodes CRUD de base. Des méthodes dérivées personnalisées ont été ajoutées : `findByStatut(Statut statut)` et `findByPriorite(Pr priorite)`, qui permettent de filtrer les incidents sans écrire de requêtes SQL explicites grâce à la convention de nommage Spring Data JPA.

1.3.3 Package service

Le package `service` contient la classe `IncidentService` qui encapsule la logique métier complexe. Cest dans cette couche que seffectuent les appels vers les autres microservices (via les clients), la validation des données, la construction des objets réponse et la gestion du mode dégradé. Elle orchestre le flux : validation MS3 → validation MS4 → persistance → notification MS5.

1.3.4 Package controller

Le package `controller` expose les endpoints REST via la classe `IncidentController`. Annotée avec `@RestController` et `@CrossOrigin` pour autoriser les appels depuis linterface web, elle reçoit les requêtes HTTP, extrait les paramètres avec `@PathVariable`, convertit le JSON en objets Java avec `@RequestBody`, et retourne des `ResponseEntity` avec les codes HTTP appropriés (201 Created, 200 OK, 404 Not Found).

1.3.5 Package dto

Le package `dto` (Data Transfer Object) contient deux classes : `IncidentRequest` pour les données entrantes (création dincident) et `IncidentResponse` pour les données sortantes. Cette séparation permet de ne pas exposer directement lentité JPA et de contrôler précisément quels champs sont reçus ou envoyés.

1.3.6 Package client

Ce package contient trois classes clientes qui encapsulent les appels HTTP vers les autres microservices en utilisant `RestTemplate` : `UtilisateurClient` (vers MS3), `EquipementClient` (vers MS4) et `NotificationClient` (vers MS5). Chaque client gère les exceptions de connexion pour activer le mode dégradé.

1.4 Base de données

La base de données `db_incidents` est propre à ce microservice et isolée des autres services. Aucun autre microservice ny accède directement. Les tables sont créées automatiquement par Hibernate au démarrage.

Colonne	**Type**	**Description**

id	BIGINT (PK)	Identifiant auto-généré
titre	VARCHAR(255)	Titre de lincident (obligatoire)
description	VARCHAR(1000)	Description détaillée
date_declaration	DATETIME	Date/heure de création (automatique)
priorite	VARCHAR(50)	FAIBLE, MOYENNE, HAUTE, CRITIQUE
categorie	VARCHAR(50)	RESEAU, MATERIEL, LOGICIEL, SECURITE, AUTRE
statut	VARCHAR(50)	OUVERT (défaut), EN_COURS, RESOLU, CLOTURE
demandeur_id	BIGINT	ID du demandeur (référence externe MS3)
demandeur_nom	VARCHAR(255)	Nom stocké localement pour affichage
equipement_id	BIGINT	ID de léquipement (référence externe MS4)
equipement_nom	VARCHAR(255)	Nom stocké localement pour affichage

1.5 API REST exposées

Notre microservice expose 6 endpoints REST essentiels accessibles sur le port 8081, tous préfixés par `/api/incidents`.

Méthode	**URL**	**Description**
POST	/api/incidents	Déclarer un nouvel incident
GET	/api/incidents	Lister tous les incidents
GET	/api/incidents/{id}	Consulter le détail dun incident
GET	/api/incidents/{id}/existe	Vérifier lexistence (usage interne MS2/MS6)
GET	/api/incidents/statut/{statut}	Filtrer par statut
GET	/api/incidents/priorite/{priorite}	Filtrer par priorité

1.6 Fonctionnalités spécifiques

1.6.1 Mode dégradé (Résilience)

Un mécanisme de mode dégradé a été implémenté. Si les microservices MS3 (Utilisateurs) ou MS4 (Équipements) sont indisponibles, le système ne bloque pas la création de lin-

cident. Il attribue des valeurs par défaut (`Utilisateur #{id}`, `Équipement #{id}`) et continue le traitement. Cela garantit la disponibilité même en cas de panne partielle.

1.6.2 Interface web intégrée

Une interface web complète a été développée en HTML5, CSS3 et JavaScript vanilla, intégrée dans le dossier `static`. Accessible à `http://localhost:8081/index.html`, elle offre : un tableau de bord visuel, un formulaire de déclaration, des filtres dynamiques et un panneau de détail.

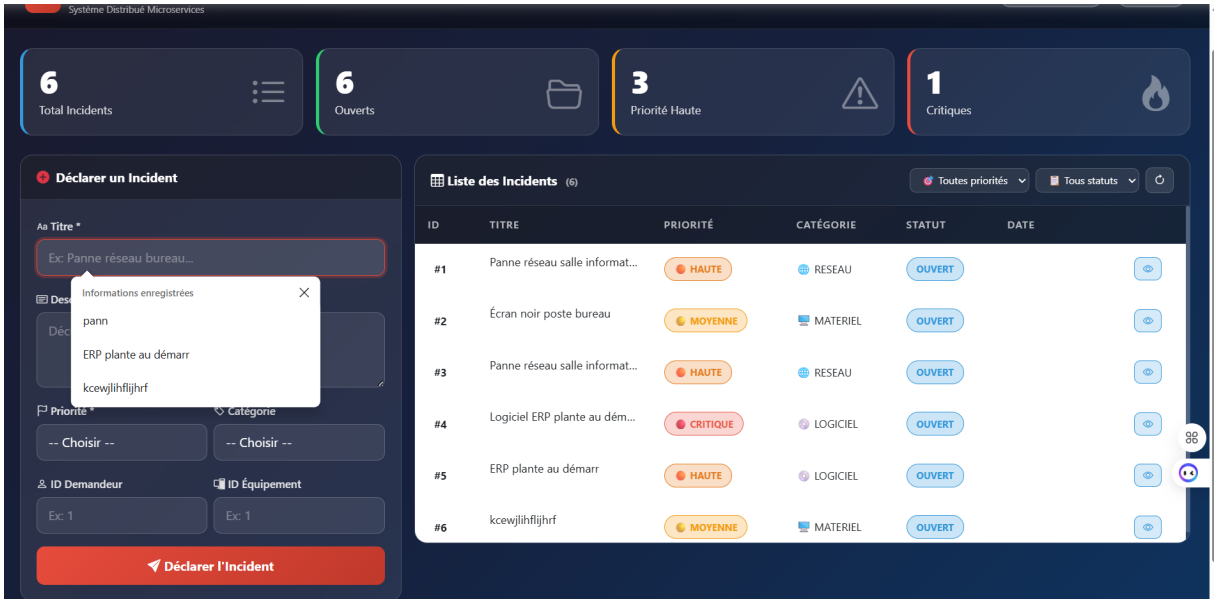

FIGURE 1.1 – Figure 1.1 – Interface web intégrée du Microservice 1 (Déclaration)

1.6.3 Documentation Swagger

La bibliothèque `springdoc-openapi` a été intégrée pour générer la documentation interactive. Accessible à ladresse suivante : `http://localhost:8081/swagger-ui.html`.

1.7 Diagramme de séquence

Le flux complet de déclaration dun incident est illustré ci-dessous :

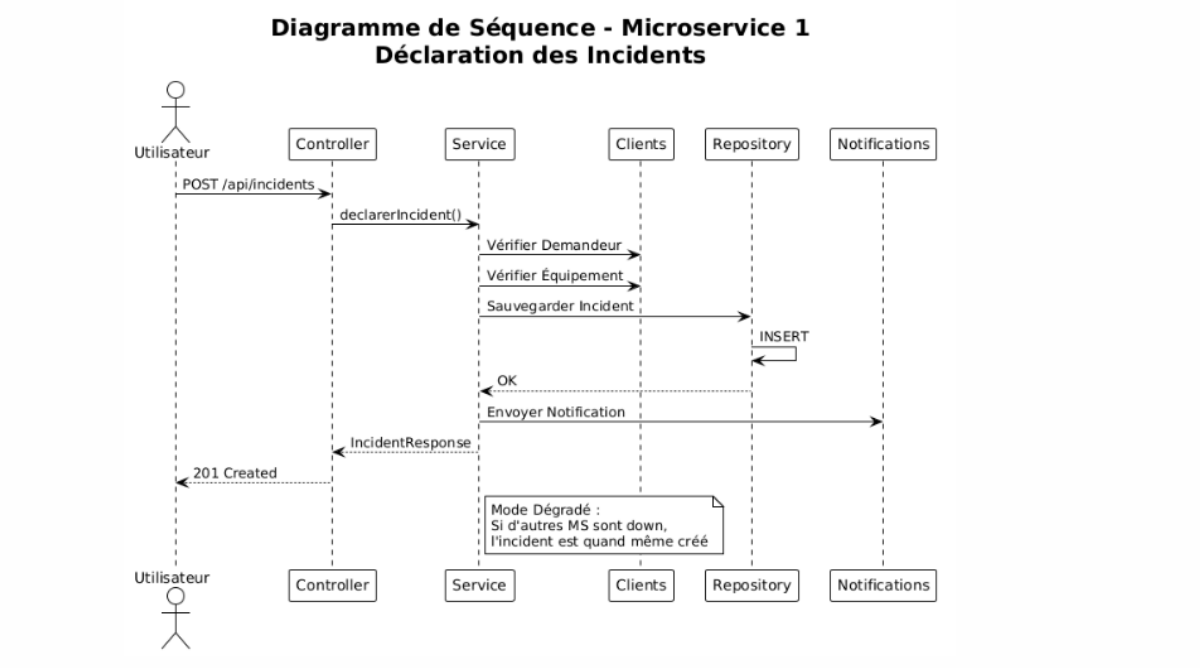

FIGURE 1.2 – Figure 1.2 – Diagramme de séquence du Microservice 1 (Déclaration)

1.8 Tests réalisés

Les scénarios validés via Postman et Swagger UI sont :

- Création dun incident : réponse 201 Created avec statut OUVERT.
- Mode dégradé (MS3/MS4 arrêtés) : création réussie avec noms par défaut.
- Validation échouée : réponse 400 Bad Request si lID est inexistant (MS3 up).
- Consultation et filtrage : succès des requêtes GET (global, ID unique, priorité).
- Incident inexistant : réponse 404 avec message derreur.

1.9 Conclusion

Le microservice Déclaration des Incidents est entièrement fonctionnel et constitue la porte dentrée opérationnelle du système distribué. Sa capacité à valider les données externes tout en restant tolérant aux pannes (mode dégradé) en fait un composant robuste et fiable, prêt à communiquer avec les microservices daffectation (MS2) et de reporting (MS6).

Chapitre 2

Microservice Affectation & Traitement

2.1 Introduction

Dans le cadre du projet de système distribué de gestion des incidents IT, chaque étudiant a développé un microservice distinct. Ma contribution porte sur le microservice Affectation & Traitement, qui constitue le cur opérationnel du cycle de traitement des incidents.

Ce microservice fonctionne sur le port 8082 et est développé avec Java 21, Spring Boot 3.2.5 et MySQL via XAMPP.

Contrairement au microservice Déclaration qui crée les incidents, ce service assure leur prise en charge technique : affectation à un technicien, suivi de lavancement, changement de statut et clôture.

2.2 Rôle dans le système distribué

Le microservice Affectation joue un rôle central dans larchitecture globale. Il intervient après la déclaration dun incident et assure son traitement complet.

Les interactions avec les autres microservices sont les suivantes :

— Le microservice Affectation appelle le MS1 (Déclaration – port 8081) via `GET /api/incidents/{id}` pour récupérer les informations dun incident avant affectation.

— Il appelle le MS3 (Utilisateurs & Équipes – port 8083) via `GET /api/utilisateurs/{id}` pour vérifier lexistence du technicien.

— Il appelle le MS5 (Notifications & Historique – port 8085) via `POST /api/notifications`

pour notifier chaque affectation ou changement de statut.

Sans ce microservice, le cycle de traitement dun incident ne peut pas être exécuté.

2.3 Architecture technique

Le microservice suit larchitecture en couches standard de Spring Boot.

2.3.1 Package model

Contient les entités :

- `Affectation` : représente laffectation dun incident.
- `SuiviTraitement` : conserve lhistorique des changements de statut.

2.3.2 Package repository

- `AffectationRepository` : étend `JpaRepository`.
- `SuiviTraitementRepository` : permet de récupérer lhistorique.

Ces interfaces fournissent automatiquement les opérations CRUD.

2.3.3 Package service

Contient la logique métier :

- Vérification de lexistence dun incident
- Vérification du technicien
- Création de laffectation
- Enregistrement du suivi
- Envoi des notifications

2.3.4 Package controller

Expose les endpoints REST :

- `POST /api/affectations`
- `GET /api/affectations`
- `PUT /api/affectations/{id}/statut`
- `GET /api/affectations/{id}/suivi`

2.3.5 Package config

— `CorsConfig` : autorise les appels inter-domaines.

2.4 Base de données

La base de données `affectation_db` est propre à ce microservice.

Table affectations

Colonne	Type	Description
id	BIGINT (PK)	Identifiant auto-généré
incident_id	BIGINT	ID de lincident (MS1)
technicien_id	BIGINT	ID du technicien (MS3)
equipe_id	BIGINT	ID de léquipe
statut	VARCHAR(50)	EN_ATTENTE, EN_COURS, RESOLU, CLOTURE
commentaire	TEXT	Description du traitement
date_affectation	DATETIME	Date de création
date_cloture	DATETIME	Date de clôture

Table suivi_traitement

Colonne	Type	Description
id	BIGINT (PK)	Identifiant auto-généré
affectation_id	BIGINT	Référence affectation
ancien_statut	VARCHAR(50)	Statut précédent
nouveau_statut	VARCHAR(50)	Nouveau statut
commentaire	TEXT	Commentaire de mise à jour
date_action	DATETIME	Date du changement

2.5 API REST exposées

Méthode	URL	Description

POST	/api/affectations	Affecter un incident
GET	/api/affectations	Lister toutes les affectations
GET	/api/affectations/{id}	Consulter une affectation
PUT	/api/affectations/{id}/statut	Changer le statut
PUT	/api/affectations/{id}/cloture	Clôturer une affectation
GET	/api/affectations/{id}/suivi	Voir lhistorique

2.6 Interface Web intégrée

Une interface web complète a été développée en HTML, CSS et JavaScript et intégrée dans le dossier `static`. Elle permet :

— laffectation dun incident ;

— la visualisation des affectations ;

— le changement de statut ;

— la consultation de lhistorique ;

— laffichage des statistiques dynamiques.

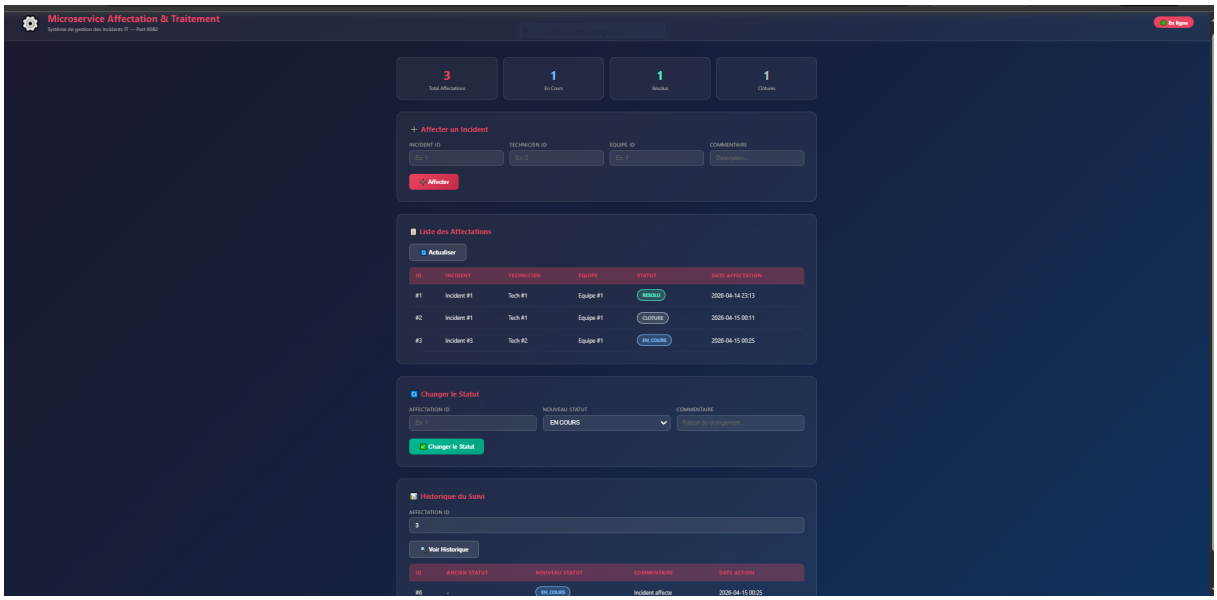

FIGURE 2.1 – Figure 1 – Interface Web du Microservice Affectation & Traitement

2.7 Diagramme de séquence

Le diagramme suivant illustre le processus complet daffectation et de changement de statut.

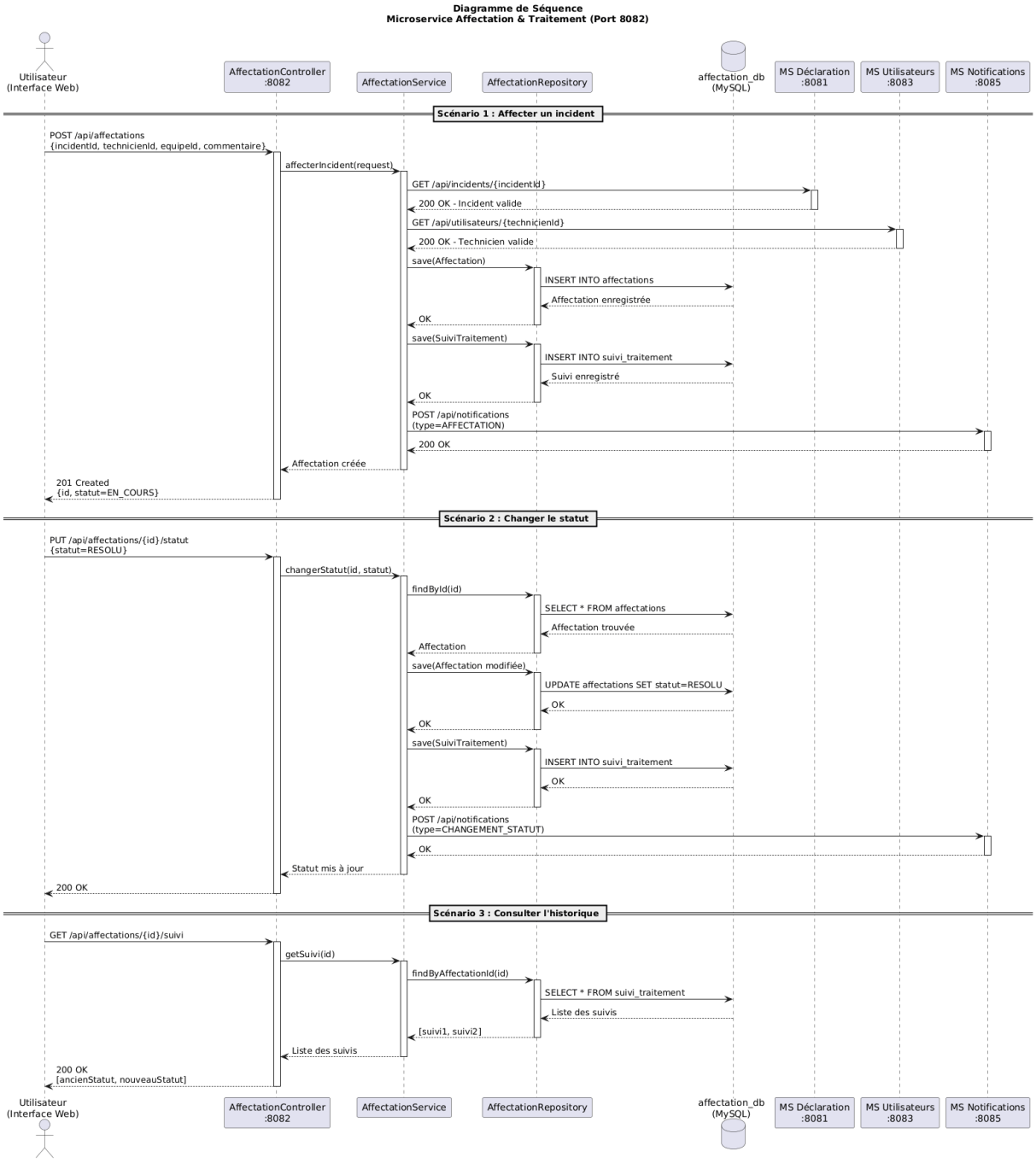

FIGURE 2.2 – Figure 2 – Diagramme de séquence du Microservice Affectation

2.8 Tests réalisés

Les tests ont été réalisés via Postman et linterface web.

Scénarios validés :

— Création dune affectation : réponse 201 Created.

— Modification du statut : réponse 200 OK.
— Consultation de lhistorique : retour correct des suivis.
— Communication inter-services validée.

2.9 Conclusion

Le microservice Affectation & Traitement est robuste, isolé et parfaitement intégré à larchitecture distribuée. Grâce à son API REST complète, sa base de données dédiée et son interface web moderne, il assure la gestion opérationnelle du traitement des incidents conformément au cahier des charges du projet.

Chapitre 3

Microservice Utilisateurs et Équipes

3.1 Introduction

Dans le cadre du projet de système distribué de gestion des incidents IT, chaque étudiant a développé un microservice distinct. Ma contribution (Ilyas Oulmidi) porte sur le microservice Utilisateurs et Équipes, qui constitue le service central de validation des identités dans larchitecture globale. Ce microservice fonctionne sur le port 8083 et est développé avec Java 21, Spring Boot et MySQL.

3.2 Rôle dans le système distribué

Le microservice Utilisateurs et Équipes joue le rôle de gardien des identités dans le système. Il est interrogé par les autres microservices pour vérifier lexistence dune personne avant toute action importante. Sans ce service, aucun incident ne peut être déclaré ni affecté.

Les services qui font appel à notre microservice sont les suivants :

- Le microservice Déclaration des incidents (port 8081) appelle `GET /utilisateurs/{id}` pour vérifier que le demandeur existe avant denregistrer un incident.
- Le microservice Affectation et traitement (port 8082) appelle `GET /techniciens/disponible` pour obtenir la liste des techniciens disponibles, puis `GET /techniciens/{id}` pour valider le technicien sélectionné.
- Le microservice Notifications et historique (port 8085) peut appeler `GET /utilisateurs/{id}` pour enrichir les traces avec le nom complet de lacteur concerné.

3.3 Architecture technique

Notre microservice suit larchitecture en couches standard de Spring Boot, organisée en cinq packages principaux.

3.3.1 Package model

Le package `model` contient les classes qui représentent les entités du système. Chaque classe est annotée avec `@Entity` et correspond à une table dans la base de données MySQL. Trois entités ont été définies :

— `Utilisateur` : représente un employé ou un administrateur du système. Ses attributs sont lidentifiant, le nom, le prénom, lemail, le téléphone, le rôle et le statut actif.

— `Technicien` : représente un technicien chargé de traiter les incidents. Il possède en plus un attribut `disponible` et une référence vers son équipe via une relation `@ManyToOne`.

— `Equipe` : représente une équipe technique regroupant plusieurs techniciens dune même spécialité. La relation avec les techniciens est de type `@OneToMany`.

3.3.2 Package repository

Le package `repository` contient les interfaces daccès à la base de données. Ces interfaces étendent `JpaRepository`, fourni par Spring Data JPA, ce qui permet dobtenir automatiquement les opérations CRUD sans écrire de requêtes SQL manuellement. Des méthodes dérivées ont été ajoutées, comme `findByDisponibleTrue()`, qui génère automatiquement la requête : `SELECT * FROM techniciens WHERE disponible = true`.

3.3.3 Package service

Le package `service` contient la logique métier de lapplication. Avant dajouter un utilisateur, le service vérifie que lemail nexiste pas déjà. Avant de modifier ou de supprimer une ressource, il vérifie que lidentifiant existe. Cette couche fait le lien entre les controllers et les repositories en appliquant les règles de lapplication.

3.3.4 Package controller

Le package `controller` expose les endpoints REST. Chaque controller est annoté avec `@RestController` et reçoit les requêtes HTTP entrantes. Il extrait les paramètres de lURL grâce à `@PathVariable` et convertit automatiquement le JSON en objets Java grâce à `@RequestBody`. Les trois controllers développés sont `UtilisateurController`, `TechnicienController` et `EquipeController`.

3.3.5 Package exception

La classe `GlobalExceptionHandler`, annotée avec `@RestControllerAdvice`, intercepte toutes les exceptions lancées dans lapplication. Au lieu de retourner une erreur technique incompréhensible, elle retourne un message JSON structuré contenant le statut, le message derreur et lhorodatage.

3.3.6 Package config

Deux classes de configuration ont été créées :

— `CorsConfig` : configure la politique CORS pour autoriser les appels HTTP depuis le navigateur et depuis les autres machines du réseau.

— `ServiceConfig` : lit les URLs des autres microservices depuis le fichier `application.properties` grâce à lannotation `@Value`, dans le cadre de la préparation cloud.

3.4 Base de données

La base de données `utilisateurs_db` est propre à ce microservice. Aucun autre service ny accède directement : toute interaction passe obligatoirement par les API REST exposées. Les tables sont créées automatiquement par Hibernate au démarrage grâce à la configuration `spring.jpa.hibernate.ddl-auto=update`.

Structure de la table utilisateurs

Colonne	Type	Description
id	BIGINT (PK)	Identifiant auto-généré
nom	VARCHAR(255)	Nom de famille
prenom	VARCHAR(255)	Prénom
email	VARCHAR(255)	Adresse email unique

telephone	VARCHAR(255)	Numéro de téléphone
role	VARCHAR(255)	ADMIN, EMPLOYE, TECHNICIEN
actif	BIT(1)	Statut actif ou inactif

Structure de la table techniciens

Colonne	Type	Description
id	BIGINT (PK)	Identifiant auto-généré
nom	VARCHAR(255)	Nom de famille
prenom	VARCHAR(255)	Prénom
email	VARCHAR(255)	Adresse email
telephone	VARCHAR(255)	Numéro de téléphone
specialite	VARCHAR(255)	Domaine de compétence
disponible	BIT(1)	Disponibilité pour affectation
equipe_id	BIGINT (FK)	Référence vers la table equipes

Structure de la table equipes

Colonne	Type	Description
id	BIGINT (PK)	Identifiant auto-généré
nom	VARCHAR(255)	Nom de léquipe
specialite	VARCHAR(255)	Spécialité technique
description	VARCHAR(255)	Description de léquipe

3.5 API REST exposées

Notre microservice expose 18 endpoints REST accessibles sur le port 8083.

Méthode	URL	Description
GET	/utilisateurs	Lister tous les utilisateurs
GET	/utilisateurs/{id}	Consulter un utilisateur par ID
POST	/utilisateurs	Ajouter un utilisateur
PUT	/utilisateurs/{id}	Modifier un utilisateur
DELETE	/utilisateurs/{id}	Supprimer un utilisateur
GET	/techniciens	Lister tous les techniciens
GET	/techniciens/{id}	Consulter un technicien par ID
GET	/techniciens/disponibles	Lister les techniciens disponibles

POST	/techniciens	Ajouter un technicien
PUT	/techniciens/{id}	Modifier un technicien
DELETE	/techniciens/{id}	Supprimer un technicien
GET	/equipes	Lister toutes les équipes
GET	/equipes/{id}	Consulter une équipe par ID
POST	/equipes	Créer une équipe
PUT	/equipes/{id}	Modifier une équipe
DELETE	/equipes/{id}	Supprimer une équipe
GET	/utilisateurs/sante	Obtenir le statut du microservice
GET	/utilisateurs/config	Consulter la configuration externalisée

Lendpoint `GET /utilisateurs/{id}` est lun des plus importants du microservice, car il est appelé directement par le microservice Déclaration des incidents pour valider chaque demandeur. De même, `GET /techniciens/disponibles` constitue un endpoint critique pour le microservice Affectation.

3.6 Fonctionnalités bonus

3.6.1 Auto-stabilisation

Un mécanisme dauto-stabilisation a été mis en place grâce à la classe `AutoStabilisationService` et à lannotation `@EnableScheduling`. Ce mécanisme exécute automatiquement trois vérifications périodiques sans intervention humaine :

— Toutes les 30 secondes : détection et correction des emails en double. Si deux utilisateurs possèdent le même email, le doublon est automatiquement désactivé.

— Toutes les 60 secondes : détection des techniciens sans équipe. Un technicien sans équipe ne pouvant pas être affecté, il est automatiquement marqué comme indisponible.

— Toutes les 2 minutes : génération dun rapport de santé dans la console, affichant les statistiques du système en temps réel.

Lannotation `@Scheduled(fixedRate = 30000)` indique à Spring dexécuter la méthode toutes les 30 secondes. Sans lannotation `@EnableScheduling` sur la classe principale, aucun timer ne démarre.

La différence fondamentale entre lauto-stabilisation et la gestion des exceptions est la suivante : la gestion des exceptions est réactive ; elle intervient lorsquune erreur survient. Lauto-stabilisation est proactive ; elle surveille et corrige en permanence, même en labsence de requêtes.

3.6.2 Préparation cloud

Les URLs des autres microservices ont été externalisées dans le fichier `application.properties` au lieu dêtre écrites en dur dans le code Java. La classe `ServiceConfig` lit ces valeurs grâce à lannotation `@Value` et les rend disponibles dans lapplication.

```
service.incidents.url=http://localhost:8081/incidents
service.affectation.url=http://localhost:8082/affectations
service.notifications.url=http://localhost:8085/notifications
service.equipements.url=http://localhost:8084/equipements
```

Si ladresse IP dun microservice change, il suffit de modifier une ligne dans ce fichier texte sans toucher au code Java ni recompiler lapplication. Cest une pratique essentielle dans un environnement cloud, où les adresses peuvent changer fréquemment.

Un endpoint dédié, `GET /utilisateurs/config`, permet de visualiser la configuration active à tout moment.

3.6.3 Interface web

Une interface web a été développée en HTML, CSS et JavaScript, puis intégrée directement dans le microservice Spring Boot via le dossier `src/main/resources/static`. Elle est accessible à ladresse `http://localhost:8083/index.html` et offre quatre fonctionnalités principales :

- un tableau de bord affichant en temps réel le nombre dutilisateurs, de techniciens et déquipes, ainsi que la liste des techniciens disponibles pour laffectation ;
- une gestion des utilisateurs avec formulaire dajout, liste complète et barre de recherche ;
- une gestion des techniciens avec sélection de léquipe et indicateur de disponibilité ;
- une gestion des équipes avec création et consultation.

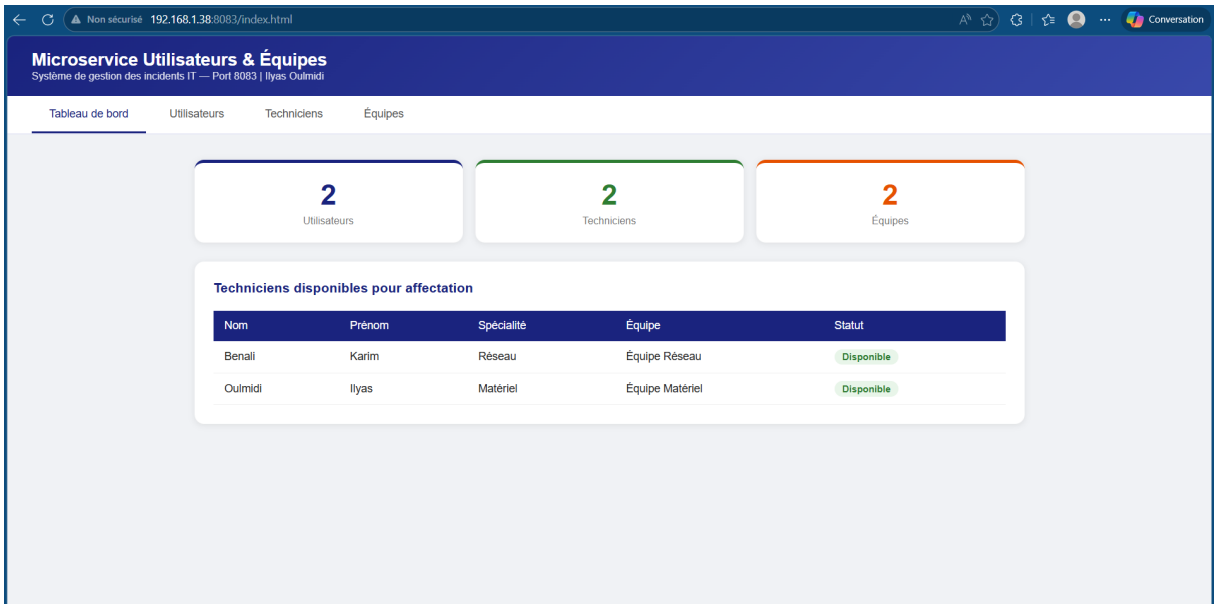

FIGURE 3.1 – Figure 1.1 – Interface web du microservice Utilisateurs et Équipes

3.6.4 Documentation Swagger

La bibliothèque `springdoc-openapi` a été intégrée pour générer automatiquement la documentation interactive de toutes les API. Elle est accessible à ladresse `http://localhost:8083/swagge` et permet de tester chaque endpoint directement depuis le navigateur, sans outil externe.

3.7 Diagramme de séquence

Le diagramme de séquence suivant illustre le flux complet de traitement dun incident dans le système distribué. Lorsquun employé déclare un incident, le microservice Déclaration contacte dabord le microservice Utilisateurs et Équipes pour vérifier que le demandeur existe, puis le microservice Équipements pour vérifier la ressource concernée. Une fois lincident enregistré, le microservice Affectation interroge à nouveau le microservice Utilisateurs et Équipes afin dobtenir la liste des techniciens disponibles et de valider le technicien sélectionné.

Ce diagramme montre que le microservice Utilisateurs et Équipes est appelé trois fois au cours dun seul cycle de traitement, ce qui confirme son rôle central dans le système.

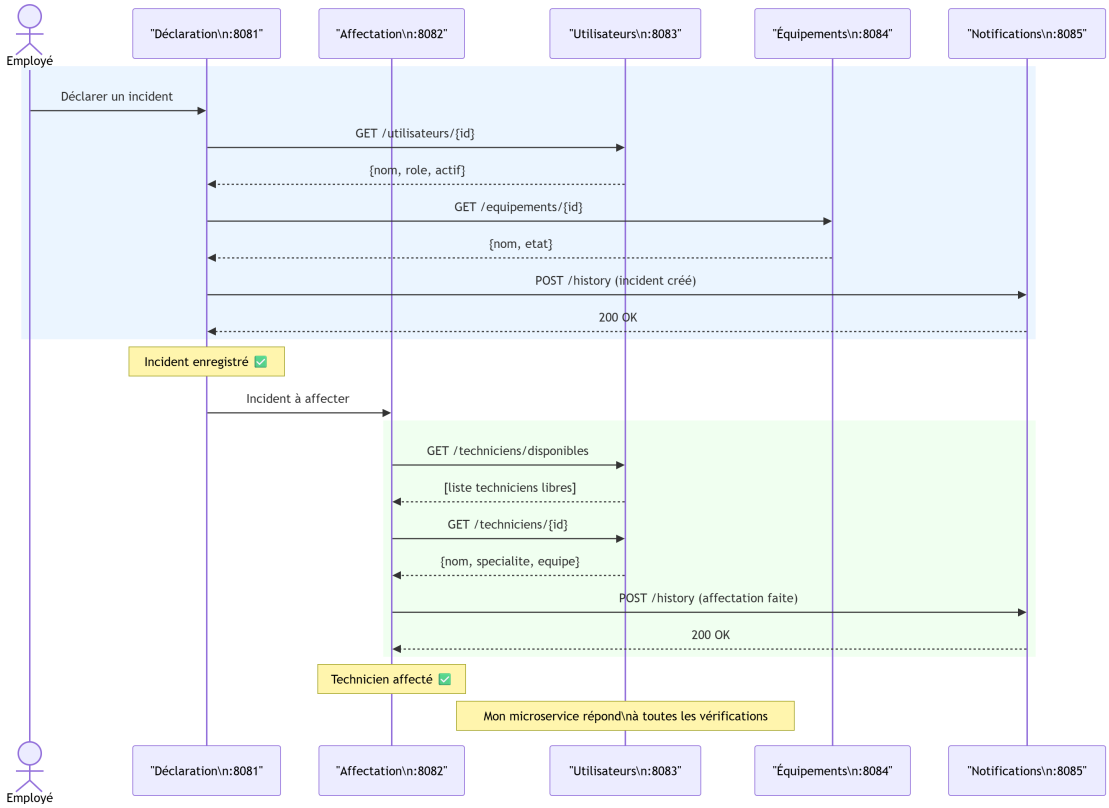

FIGURE 3.2 – Figure 1.2 – Diagramme de séquence du traitement dun incident dans le système distribué

3.8 Tests réalisés

Les tests ont été réalisés avec Postman et Swagger UI. Les scénarios validés sont les suivants :

1. Création dune équipe via `POST /equipes` : réponse 200 OK avec lobjet créé.
2. Création dun technicien lié à une équipe via `POST /techniciens` : réponse 200 OK.
3. Création dun utilisateur via `POST /utilisateurs` : réponse 200 OK.
4. Consultation dun utilisateur existant via `GET /utilisateurs/1` : réponse 200 OK avec les données.
5. Consultation dun ID inexistant via `GET /utilisateurs/999` : réponse 400 avec le message ń Utilisateur avec ID 999 introuvable ż.
6. Liste des techniciens disponibles via `GET /techniciens/disponibles` : réponse 200 OK.
7. Test de lauto-stabilisation : ajout dun technicien sans équipe, puis correction automatique après 60 secondes.

8. Test de la configuration cloud via `GET /utilisateurs/config` : affichage des URLs externalisées.

3.9 Conclusion

Le microservice Utilisateurs et Équipes est entièrement fonctionnel et prêt à être intégré dans le système distribué global. Il expose 18 endpoints REST documentés, dispose dune base de données indépendante, dune interface web complète, dun mécanisme dautostabilisation, dune préparation cloud et dune documentation Swagger automatique. Son rôle de service central de validation en fait un composant indispensable au fonctionnement cohérent de lensemble du système.

Chapitre 4

Microservice Équipements et Ressources IT

4.1 Introduction

Dans le cadre du projet de système distribué de gestion des incidents IT, chaque étudiant a développé un microservice distinct. Ma contribution (Bilal EL-Haoudar) porte sur le microservice Équipements et Ressources IT, qui constitue le référentiel matériel de larchitecture globale. Ce microservice fonctionne sur le port 8084 et est développé avec Java 17, Spring Boot et MySQL.

4.2 Rôle dans le système distribué

Le microservice Équipements joue le rôle de gestionnaire du parc informatique. Il est interrogé par les autres microservices pour vérifier lexistence, le type et létat du matériel avant toute intervention technique. Sans ce service, il est impossible de lier un incident à une ressource physique.

Les services qui font appel à notre microservice sont les suivants :

— Le microservice Déclaration des incidents (port 8081) appelle `GET /equipements/{id}` pour vérifier que léquipement concerné existe réellement et récupérer son nom avant denregistrer le ticket.

— Le microservice Affectation et traitement (port 8082) le consulte pour obtenir le contexte technique et la catégorie du matériel en panne afin de laffecter au bon technicien.

— Le microservice Notifications et historique (port 8085) reçoit des événements de notre service pour tracer lhistorique de vie du matériel (ex : passage dun serveur

en statut EN_PANNE).

4.3 Architecture technique

Notre microservice suit larchitecture en couches standard de Spring Boot, organisée en plusieurs packages principaux :

4.3.1 Package model

Contient lentité `Equipement` annotée avec `@Entity`, correspondant à la table dans la base de données MySQL. Ses attributs sont lidentifiant (généré automatiquement), le nom, la description technique, la catégorie (Serveur, Routeur, PC, etc.) et létat actuel (ACTIF, EN_MAINTENANCE, EN_PANNE).

4.3.2 Package repository

Contient linterface `EquipementRepository` qui étend `JpaRepository`. Cette interface fournit automatiquement les opérations CRUD. Une méthode personnalisée `findByCategorie(String categorie)` a été ajoutée pour filtrer le matériel dynamiquement sans écrire de requêtes SQL manuellement.

4.3.3 Package service

Contient la logique métier de lapplication. Avant de modifier létat dun équipement, le service sassure que lidentifiant existe. Il prépare également les données pour les notifications inter-services.

4.3.4 Package controller

Expose les endpoints REST. Le `EquipementController` (annoté avec `@RestController`) reçoit les requêtes HTTP, extrait les paramètres via `@PathVariable` et convertit le JSON en objets Java via `@RequestBody`.

4.3.5 Packages exception et config

— `GlobalExceptionHandler` : intercepte les erreurs (comme un ID introuvable) pour retourner un message JSON structuré au lieu dune erreur technique serveur.

— `CorsConfig` : configure la politique CORS pour autoriser les appels HTTP depuis linterface web React (Dashboard).
— `ServiceConfig` : lit les URLs des autres microservices depuis `application.properties` dans le cadre de la préparation Cloud.

4.4 Base de données

La base de données `equipements_db` est propre à ce microservice. Strictement aucun autre service ny accède directement ; toute interaction passe par les API REST.

Colonne	Type	Description
id	BIGINT (PK)	Identifiant auto-généré
nom	VARCHAR(255)	Nom de léquipement (ex : Serveur Dell)
description	VARCHAR(255)	Détails techniques de la ressource
categorie	VARCHAR(255)	Type de matériel (Serveur, PC, Routeur)
etat	VARCHAR(50)	ACTIF, EN_MAINTENANCE, EN_PANNE

4.5 API REST exposées

Notre microservice expose 5 endpoints REST principaux accessibles sur le port 8084.

Méthode	URL	Description
GET	/equipements	Lister tous les équipements IT
GET	/equipements/{id}	Consulter un équipement précis par ID
GET	/equipements/categorie/{cat}	Filtrer les équipements par catégorie
POST	/equipements	Ajouter un nouvel équipement
PUT	/equipements/{id}/etat	Mettre à jour létat dune ressource

4.6 Fonctionnalités bonus

4.6.1 Tableau de bord centralisé (Interface Web React)

Une interface web complète de type ń Single Page Application ż a été développée en React.js et Tailwind CSS. Elle ne se limite pas à mon microservice, mais centralise les API de tous les microservices du groupe. Elle offre :

— une vue densemble (Dashboard) avec un ń Ping ż de santé pour vérifier létat (Online/Offline) des autres microservices de léquipe ;
— la gestion complète des équipements avec statistiques en temps réel ;
— une configuration dynamique des URLs via `localStorage` pour sadapter aux changements dadresses IP des différentes machines du groupe sur le réseau local.

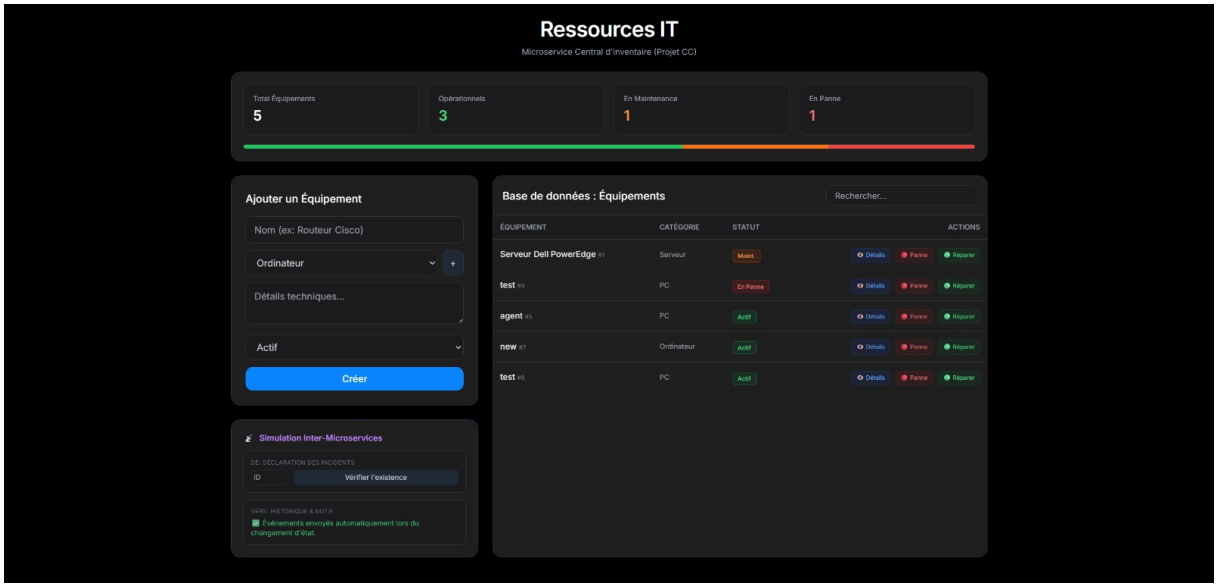

FIGURE 4.1 – Figure 1.1 – Dashboard React : Supervision globale du système distribué

4.6.2 Préparation Cloud et Auto-stabilisation

Les URLs des services dépendants ont été externalisées dans `application.properties`. De plus, une tâche planifiée (`@Scheduled`) vérifie périodiquement la cohérence des états du matériel pour éviter les ń faux positifs ż de pannes prolongées.

4.6.3 Documentation Swagger

Lintégration de `springdoc-openapi` génère automatiquement la documentation interactive des API, facilitant grandement les tests dintégration par les autres membres du groupe.

4.7 Conception et Diagrammes

4.7.1 Diagramme de classes

Le diagramme suivant montre la structure de notre entité métier et son intégration avec les interfaces de données et les contrôleurs REST.

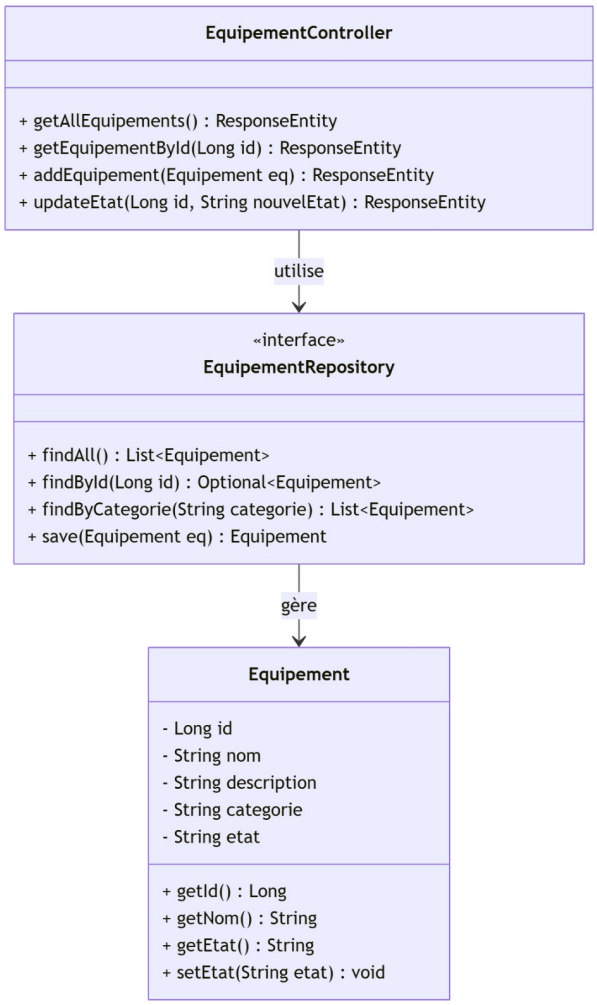

FIGURE 4.2 – Figure 1.2 – Diagramme de classes du microservice Équipements

4.7.2 Diagramme de séquence

Le diagramme de séquence illustre le traitement dune mise à jour détat dun équipement, incluant la communication avec la base de données locale et la notification asynchrone aux autres services du système distribué.

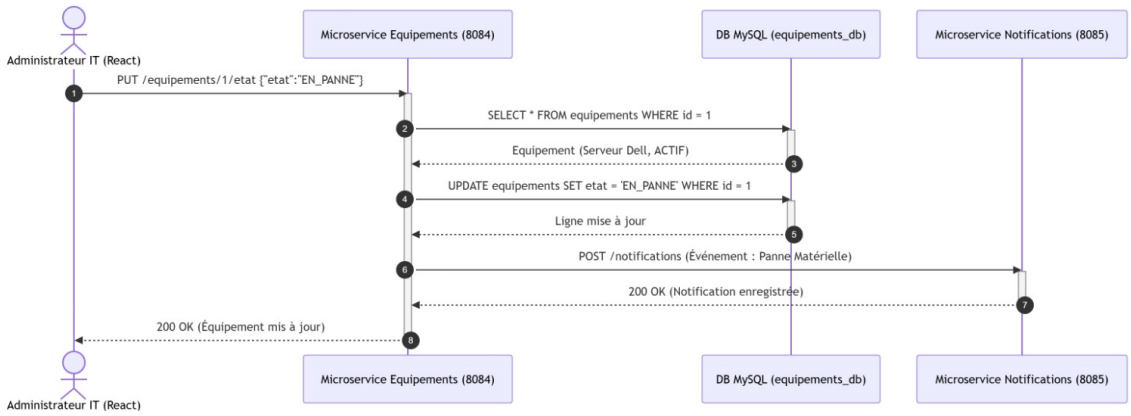

FIGURE 4.3 – Figure 1.3 – Diagramme de séquence : Signalement dune panne

4.8 Tests réalisés

Les tests dintégration ont été réalisés avec succès via Postman (Collection : `CC-spring-boot-test-API`). Les scénarios validés sont :

1. Ajout déquipement via `POST /equipements` : création dun ń Serveur Dell PowerEdge ż avec statut ACTIF (réponse 200 OK).
2. Lecture globale via `GET /equipements` : récupération de la liste complète du parc informatique.
3. Consultation unitaire via `GET /equipements/1` : récupération des détails du serveur ajouté.
4. Mise à jour via `PUT /equipements/1/etat` : modification de létat du matériel en utilisant un payload JSON partiel.
5. Filtrage métier via `GET /equipements/categorie/Serveur` : validation de la méthode dérivée du Repository.

4.9 Conclusion

Le microservice Équipements et Ressources IT est robuste, isolé et parfaitement intégré à lécosystème distribué. Grâce à son API documentée, sa base de données dédiée et son tableau de bord React innovant, il assure la gestion complète et en temps réel du parc matériel, répondant ainsi à toutes les exigences du cahier des charges.

Chapitre 5

Microservice Notifications et Historique

5.1 Introduction

Dans le cadre du projet de système distribué de gestion des incidents IT, chaque étudiant a développé un microservice distinct. Ma contribution porte sur le microservice Notifications et Historique, qui constitue la couche transversale de traçabilité et de communication de larchitecture globale. Ce microservice fonctionne sur le port 8084 et est développé avec Java 17, Spring Boot 3 et MySQL.

Ce service est le point de convergence de tous les événements importants du système. Chaque action effectuée par les autres microservices – déclaration dun incident, affectation à un technicien, changement détat, vérification déquipement – génère un événement qui doit être enregistré et notifié de manière permanente.

5.2 Rôle dans le système distribué

Le microservice Notifications et Historique joue le rôle de registre central du système. Il est alimenté par les autres microservices et constitue la mémoire collective de toutes les actions effectuées. Sans ce service, aucun audit ni aucune traçabilité ne serait possible dans lapplication.

Les services qui font appel à notre microservice sont les suivants :

— Le microservice Déclaration des incidents (port 8081) appelle `POST /notifications/evenement` dès quun incident est créé, afin denregistrer lévénement dans lhistorique et de déclencher une notification.

— Le microservice Affectation et traitement (port 8082) appelle `POST /notifications/evenement` à chaque affectation dun technicien ou changement détat dun incident.
— Le microservice Équipements et ressources IT (port 8085) appelle `POST /notifications/histori` directement pour enregistrer une vérification déquipement dans lhistorique.

Notre microservice consomme également un service externe :

— Le microservice Utilisateurs et Équipes (port 8083) est interrogé via `GET /users/{id}` pour enrichir les entrées dhistorique avec le nom complet de lacteur concerné.

5.3 Architecture technique

Notre microservice suit larchitecture en couches standard de Spring Boot, organisée en six packages principaux.

5.3.1 Package model

Le package `model` contient les classes qui représentent les entités du système. Chaque classe est annotée avec `@Entity` et correspond à une table dans la base de données MySQL. Deux entités ont été définies :

— `Notification` : représente une notification générée automatiquement à la suite dun événement. Ses attributs sont lidentifiant, lidentifiant de lincident, le type, le message, le destinataire et la date denvoi.
— `Historique` : représente une entrée dans lhistorique des actions. Elle contient lidentifiant, lidentifiant de lincident, laction effectuée, les détails, lidentifiant de lacteur, le nom enrichi de lacteur et la date de laction.

5.3.2 Package dto

Le package `dto` contient la classe `EvenementDTO`, qui est lobjet de transfert reçu des autres microservices. Il encapsule lidentifiant de lincident, le type dévénement, les détails et lidentifiant de lacteur. Ce DTO ne touche jamais la base de données directement.

5.3.3 Package repository

Le package `repository` contient les interfaces daccès à la base de données. Ces interfaces étendent `JpaRepository`, fourni par Spring Data JPA, ce qui permet dobtenir automatiquement les opérations CRUD sans écrire de requêtes SQL manuellement. Des méthodes

dérivées ont été ajoutées, comme `findByIncidentIdOrderByDateActionDesc()`, qui génère automatiquement une requête triée par date décroissante.

5.3.4 Package service

Le package `service` contient la logique métier de lapplication. Lorsquun événement est reçu, le service :

1. interroge le microservice Utilisateurs pour récupérer le nom de lacteur via `RestTemplate` ;
2. crée et persiste une entrée dans la table `historique` ;
3. génère et persiste une notification dans la table `notification`.

En cas dindisponibilité du service Utilisateurs, le traitement continue avec le nom générique ń Inconnu ż grâce à un bloc `try/catch`.

5.3.5 Package controller

Le package `controller` expose les endpoints REST. Le `NotificationController` est annoté avec `@RestController` et `@RequestMapping("/notifications")`. Il délègue entièrement la logique au service sans contenir de règles métier.

5.3.6 Package config

La classe `AppConfig` déclare le bean `RestTemplate` qui permet au service dappeler le microservice Utilisateurs via HTTP. Ce bean est injecté dans le service grâce à lannotation `@RequiredArgsConstructor` de Lombok.

5.4 Base de données

La base de données `notification_db` est propre à ce microservice. Aucun autre service ny accède directement : toute interaction passe obligatoirement par les API REST exposées. Les tables sont créées automatiquement par Hibernate au démarrage grâce à la configuration `spring.jpa.hibernate.ddl-auto=update`.

Structure de la table notification

Colonne	Type	Description

id	BIGINT	Identifiant auto-généré (PK)
incident_id	BIGINT	Référence à lincident concerné
type	VARCHAR	CREATION, AFFECTATION, CHANGEMENT_ETAT, EQUIPEMENT_VERIFIE
message	VARCHAR	Message généré automatiquement selon le type dévénement
destinataire	VARCHAR	Destinataire de la notification
date_envoi	DATETIME	Horodatage automatique via `@CreationTimestamp`

Structure de la table historique

Colonne	Type	Description
id	BIGINT	Identifiant auto-généré (PK)
incident_id	BIGINT	Référence à lincident concerné
action	VARCHAR	Type daction enregistrée (INCIDENT_CREE, AFFECTE, etc.)
details	TEXT	Description détaillée de laction effectuée
acteur_id	BIGINT	Identifiant de lutilisateur responsable de laction
acteur_nom	VARCHAR	Nom enrichi via appel GET /users/{id} au microservice Utilisateurs
date_action	DATETIME	Horodatage automatique via `@CreationTimestamp`

5.5 API REST exposées

Notre microservice expose 5 endpoints REST accessibles sur le port 8084.

Méthode	URL	Description
POST	/notifications/evenement	Recevoir et traiter un événement – crée simultanément une notification et une entrée historique

POST	/notifications/historique	Ajouter directement une entrée historique – utilisé par le microservice Équipements
GET	/notifications	Lister toutes les notifications enregistrées dans le système
GET	/notifications/historique	Consulter lhistorique complet de toutes les actions du système
GET	/notifications/historique/incident/{id}	Consulter lhistorique filtré par identifiant dincident – trié par date décroissante

Lendpoint `POST /notifications/evenement` est le plus critique du microservice, car il est appelé directement par les microservices Déclaration et Affectation à chaque événement important. De même, `GET /notifications/historique/incident/{id}` constitue lendpoint daudit principal du système.

5.6 Diagrammes de séquence

Le diagramme de séquence suivant illustre le flux complet de traitement dun événement dans notre microservice. Deux scénarios sont présentés : la réception dun événement envoyé par le microservice Déclaration (flux principal), et lajout direct dune entrée historique par le microservice Équipements (flux secondaire).

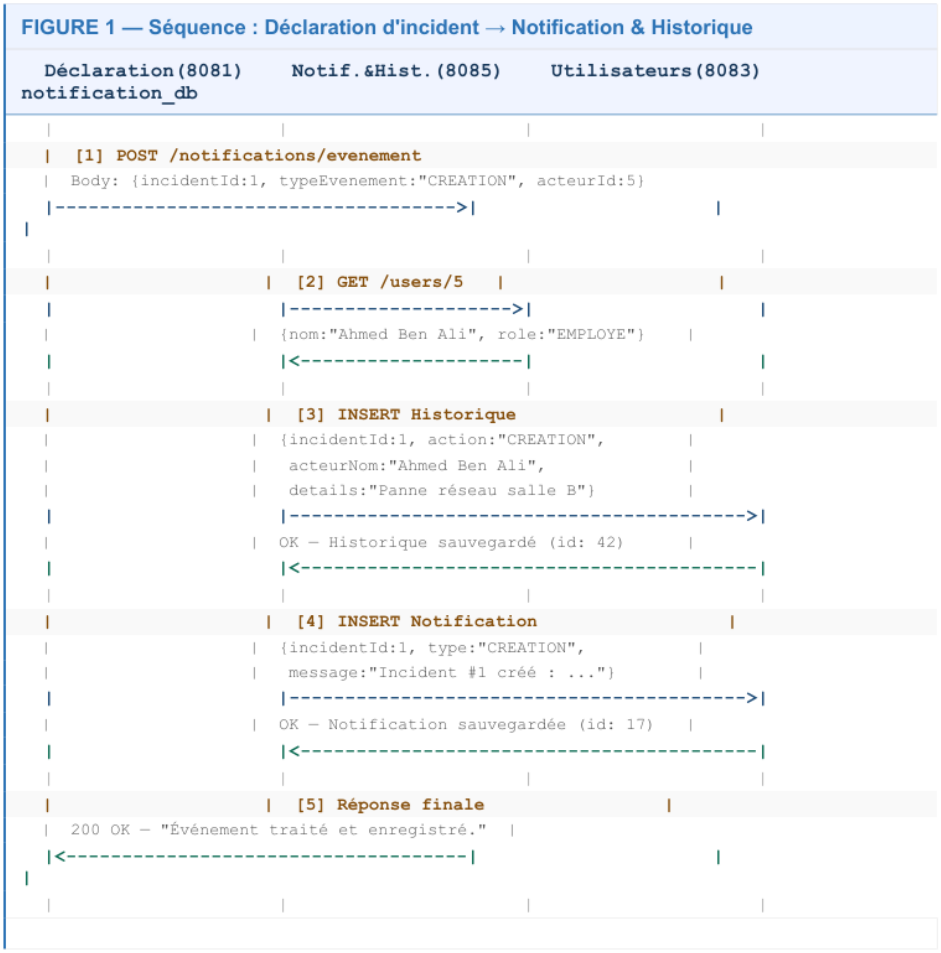

Figure 1 — Diagramme de séquence : réception d'un événement depuis le microservice Déclaration (port 8081)

FIGURE 5.1 – Figure 1 – Séquence : création dincident vers notification & historique

Ce diagramme montre que le microservice Notifications et Historique est le seul service qui écrit dans la base `notification_db`. Il réalise deux insertions consécutives à chaque événement reçu : dabord dans la table historique, puis dans la table notification. Le microservice Utilisateurs est interrogé entre les deux pour enrichir le nom de lacteur.

Figure 2 — Diagramme de séquence : ajout direct d'une entrée historique depuis le microservice Équipements (port 8084)

FIGURE 5.2 – Figure 2 – Séquence : ajout direct dune entrée historique depuis le microservice Équipements

Dans ce flux, le microservice Équipements ne passe pas par le mécanisme dévénement – il envoie directement un objet `Historique` complet, y compris le nom de lacteur. Aucun appel au microservice Utilisateurs nest nécessaire dans ce cas. Lobjet est persisté directement en base et retourné avec son identifiant généré.

5.7 Tests réalisés

Les tests ont été réalisés avec Postman. Les scénarios validés sont les suivants :

1. Envoi dun événement CREATION via `POST /notifications/evenement` : réponse 200 avec le message ń Événement traité et enregistré. ż.
2. Consultation de toutes les notifications via `GET /notifications` : réponse 200 avec la liste JSON des notifications créées.
3. Ajout dune entrée historique directement via `POST /notifications/historique` (simulation du microservice Équipements) : réponse 200 avec lobjet Historique complet incluant lidentifiant généré.

4. Consultation de lhistorique dun incident précis via `GET /notifications/historique/incident/` réponse 200 avec la liste triée par date décroissante.
5. Test de résilience : arrêt du microservice Utilisateurs, puis envoi dun événement – le traitement se poursuit avec le nom ń Inconnu ż, sans interruption du service.

5.8 Conclusion

Le microservice Notifications et Historique est entièrement fonctionnel et prêt à être intégré dans le système distribué global. Il expose 5 endpoints REST, dispose dune base de données indépendante, dun mécanisme de résilience en cas dindisponibilité des services dépendants, et dun système denrichissement automatique des données via le microservice Utilisateurs.

Son rôle de registre central de traçabilité en fait un composant indispensable au fonctionnement cohérent de lensemble du système. Ce diagramme montre que le microservice Notifications et Historique est alimenté par trois microservices différents (Déclaration, Affectation, Équipements) et interroge lui-même le microservice Utilisateurs pour enrichir les données, ce qui confirme son rôle de pivot transversal dans larchitecture distribuée.

Conformément aux règles de larchitecture microservices : aucune table partagée, aucun accès direct aux bases des autres services. Toute communication passe exclusivement par les API REST exposées.

Chapitre 6

Schéma des microservices et description de larchitecture globale

6.1 Vue densemble

Larchitecture globale du projet repose sur cinq microservices spécialisés, chacun responsable dun sous-domaine précis du cycle de traitement des incidents IT :

1. **Déclaration des Incidents** : point dentrée du système, responsable de la création des tickets et des validations initiales.
2. **Affectation & Traitement** : prise en charge technique des incidents, affectation à un technicien, suivi, changement de statut et clôture.
3. **Utilisateurs & Équipes** : gestion des identités, techniciens et équipes, service de validation pour les autres microservices.
4. **Équipements & Ressources IT** : référentiel matériel, validation de léquipement concerné, contexte technique et état des ressources.
5. **Notifications & Historique** : registre transverse, traçabilité, notifications et audit de toutes les actions importantes.

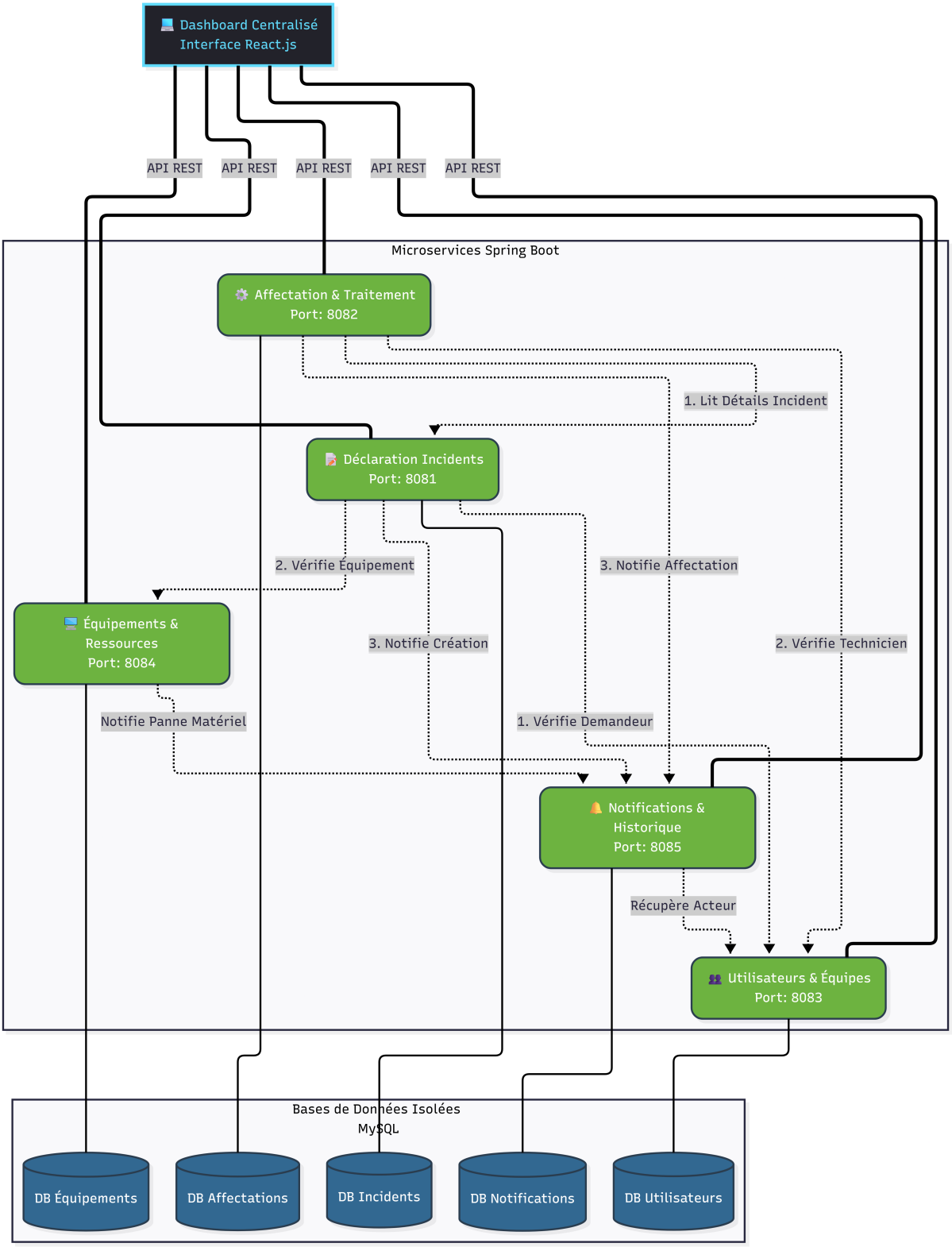

FIGURE 6.1 – Schéma global des microservices du projet

6.2 Ordre logique du flux métier

Le traitement global dun incident peut être résumé comme suit :

1. Un utilisateur déclare un incident via le microservice Déclaration.
2. Le microservice Déclaration valide le demandeur auprès du microservice Utilisateurs & Équipes.
3. Le microservice Déclaration valide léquipement concerné auprès du microservice Équipements & Ressources IT.
4. Si les validations sont correctes, lincident est persisté dans la base du microservice Déclaration.
5. Le microservice Déclaration notifie le microservice Notifications & Historique pour tracer la création.
6. Ensuite, le microservice Affectation & Traitement récupère lincident, vérifie le technicien auprès du microservice Utilisateurs & Équipes et enregistre laffectation.
7. À chaque changement de statut ou événement important, le microservice Notifications & Historique met à jour la mémoire collective du système.

6.3 Description de larchitecture globale

Larchitecture est volontairement distribuée. Chaque microservice possède :

— sa propre base de données ;
— son propre modèle métier ;
— sa propre logique métier ;
— ses propres endpoints REST ;
— sa propre interface de démonstration ou son propre support de test.

Cette séparation présente plusieurs avantages :

— meilleure modularité ;
— maintenance simplifiée ;
— déploiement indépendant ;
— test plus simple de chaque brique ;
— plus grande tolérance aux pannes partielles.

Toutefois, elle implique également une rigueur forte dans la gestion des appels inter-services. Chaque microservice doit connaître lURL des services dont il dépend, gérer les cas derreur, et maintenir la cohérence des données sans accès direct à la base dun autre service.

6.4 Convention de ports retenue pour lintégration globale

Afin dharmoniser la version finale du projet, la convention suivante est retenue pour la lecture globale du système :

Microservice	**Port cible**	**Rôle**
Déclaration des Incidents	8081	Point dentrée, création et consultation des incidents
Affectation & Traitement	8082	Affectation, suivi, changement de statut, clôture
Utilisateurs & Équipes	8083	Validation des identités, techniciens et équipes
Équipements & Ressources IT	8084	Référentiel matériel et état des ressources
Notifications & Historique	8085	Traçabilité, notifications et audit

Remarque importante : certains rapports individuels fournis comportent des notations de ports différentes (notamment pour les microservices Équipements et Notifications). Le présent chapitre fixe la convention harmonisée à utiliser pour la version globale finale, sans modifier le contenu original des chapitres individuels.

Chapitre 7

Répartition claire du travail

7.1 Principe de répartition

Le travail a été réparti par microservice, chaque étudiant étant responsable dune brique fonctionnelle clairement identifiée. Cette répartition a permis de développer les composants en parallèle tout en gardant une cohérence densemble grâce aux conventions communes : Spring Boot, JPA, bases indépendantes, API REST, documentation et tests.

7.2 Répartition des microservices

Microservice	**Responsable**	**Travail principal réalisé**
Déclaration des Incidents	Ilyas Choqri	Point dentrée du système, création dincidents, validations externes initiales, mode dégradé, interface web intégrée, Swagger
Affectation & Traitement	Ahmed Nahri	Affectation des incidents, suivi de traitement, changement de statut, clôture, historique des suivis, interface web intégrée

Utilisateurs & Équipes	Ilyas Oulmidi	Gestion des utilisateurs, techniciens et équipes, validation des identités, auto-stabilisation, configuration cloud, interface web et Swagger
Équipements & Ressources IT	Bilal EL-Haoudar	Gestion du parc matériel, validation des équipements, mise à jour détat, tableau de bord React, préparation cloud et auto-stabilisation
Notifications & Historique	Anas Hemri	Historisation des événements, génération de notifications, enrichissement des acteurs via le microservice Utilisateurs, résilience et audit

7.3 Responsabilités transversales

Au-delà du développement individuel, plusieurs tâches transversales ont dû être coordonnées à léchelle du groupe :

- harmonisation des ports et des URLs inter-services ;
- définition des contrats dAPI entre microservices ;
- préparation des jeux de données communs (utilisateurs, techniciens, équipes, équipements, incidents) ;
- vérification des scénarios inter-services via Postman, Swagger et interfaces web ;
- préparation de linterface globale de démonstration et de la soutenance.

7.4 Synthèse

La répartition du travail repose donc sur une logique claire : une responsabilité fonctionnelle par étudiant, complétée par une coordination collective sur lintégration finale. Cette organisation reflète bien les objectifs du module, qui ne demandent pas cinq applications indépendantes, mais un véritable système distribué cohérent.

Chapitre 8

Réalisation de linterface finale

8.1 Introduction

Dans la phase finale du projet, une interface web centralisée a été réalisée afin de regrouper visuellement les fonctionnalités des différents microservices du système distribué de gestion des incidents IT. Cette interface joue le rôle de portail global de supervision et daccès aux services métiers.

Lobjectif de cette réalisation est double :

— offrir une vue densemble du système distribué depuis une seule interface ;

— permettre à lutilisateur final daccéder facilement aux fonctionnalités de chaque microservice sans devoir ouvrir plusieurs applications séparées.

Linterface finale repose sur une organisation en menu latéral avec plusieurs modules : *Overview*, *Incidents*, *Affectations*, *Equipements*, *Utilisateurs*, *Logs* et *Settings*. Elle centralise également les informations de santé des services, les compteurs métier et les paramètres de connexion aux microservices du groupe.

8.2 Vue densemble du système

La page *Overview* constitue le tableau de bord principal de linterface finale. Elle présente un résumé global du système distribué avec les indicateurs principaux, létat de disponibilité des microservices, une activité récente et des informations système générales.

Cette page permet à lutilisateur ou à ladministrateur davoir immédiatement une vision synthétique de létat global de larchitecture distribuée.

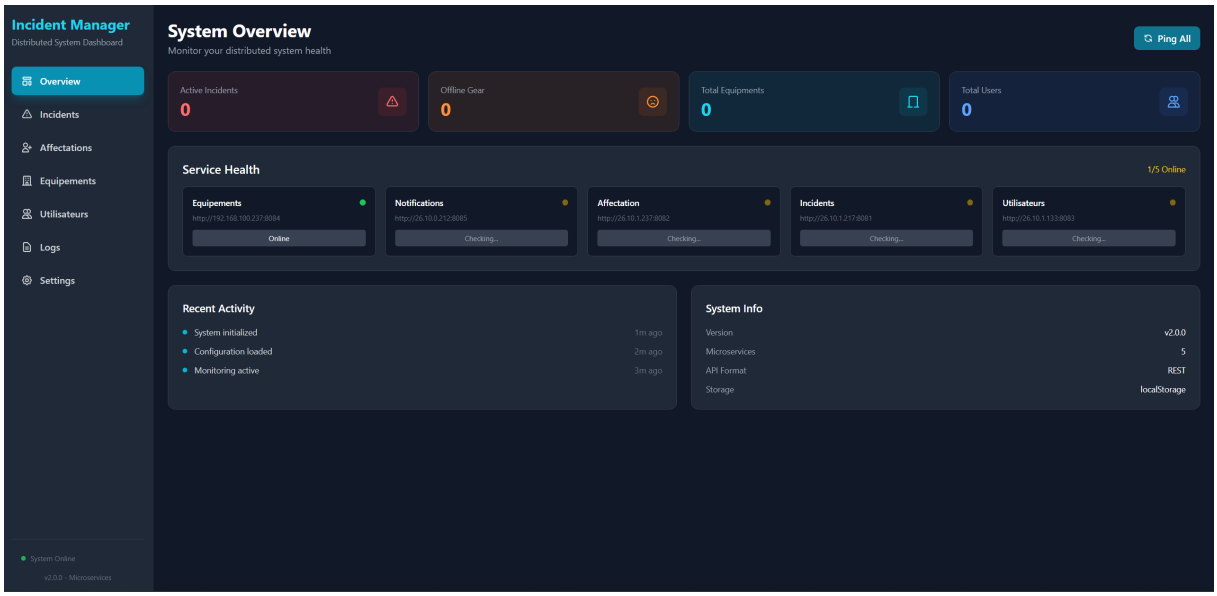

FIGURE 8.1 – Figure – Interface finale : vue densemble du système distribué

Description : cette interface affiche les indicateurs principaux du système, létat de santé des microservices, le nombre de services en ligne, ainsi quun résumé des activités récentes et des informations générales de supervision.

8.3 Module de déclaration des incidents

Le module *Incidents* correspond à lintégration visuelle du microservice de déclaration des incidents. Il permet de consulter les tickets déclarés, de filtrer les incidents par priorité, et daccéder à la création dun nouveau ticket.

Dans la capture fournie, on observe également un message derreur de connexion lorsque le service nest pas disponible, ce qui montre que linterface globale gère aussi les cas dindisponibilité dun microservice.

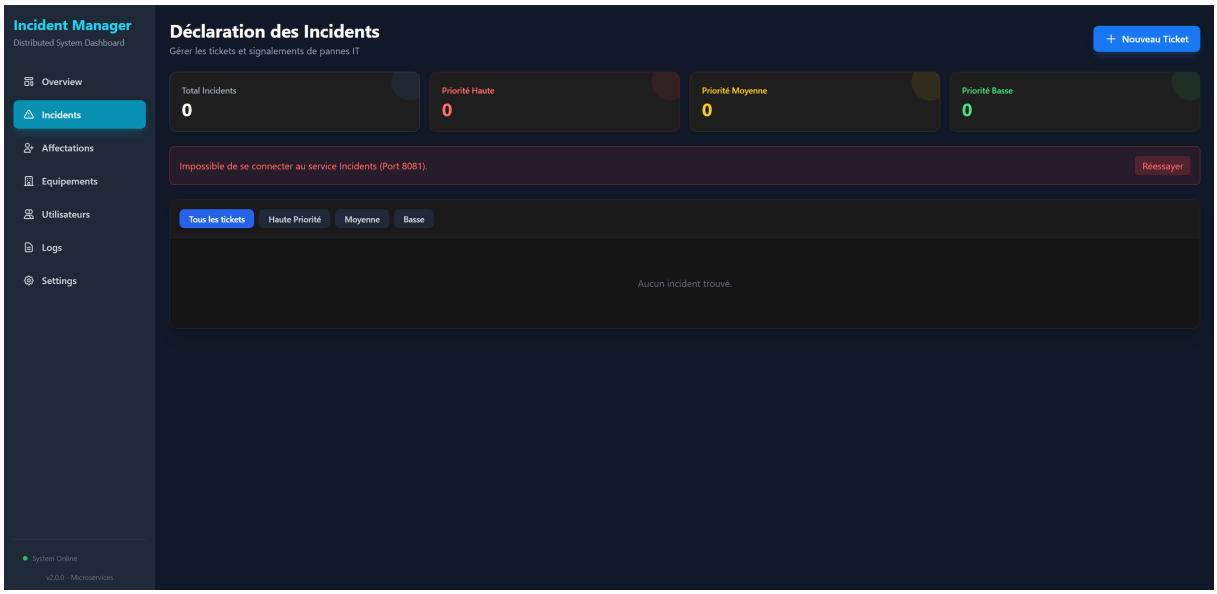

FIGURE 8.2 – Figure – Interface finale : module de déclaration des incidents

Description : cette vue centralise la gestion des incidents déclarés, laffichage des statistiques par priorité et la création de nouveaux tickets, tout en affichant un message explicite lorsque le microservice correspondant est inaccessible.

8.4 Module daffectation et de traitement

Le module *Affectations* regroupe les fonctionnalités du microservice daffectation et de traitement. Il permet de visualiser les affectations existantes, leur statut, ainsi que les compteurs liés aux traitements en cours, résolus ou clôturés.

Comme pour la partie incidents, linterface prévoit un message derreur si le service daffectation nest pas joignable sur le réseau.

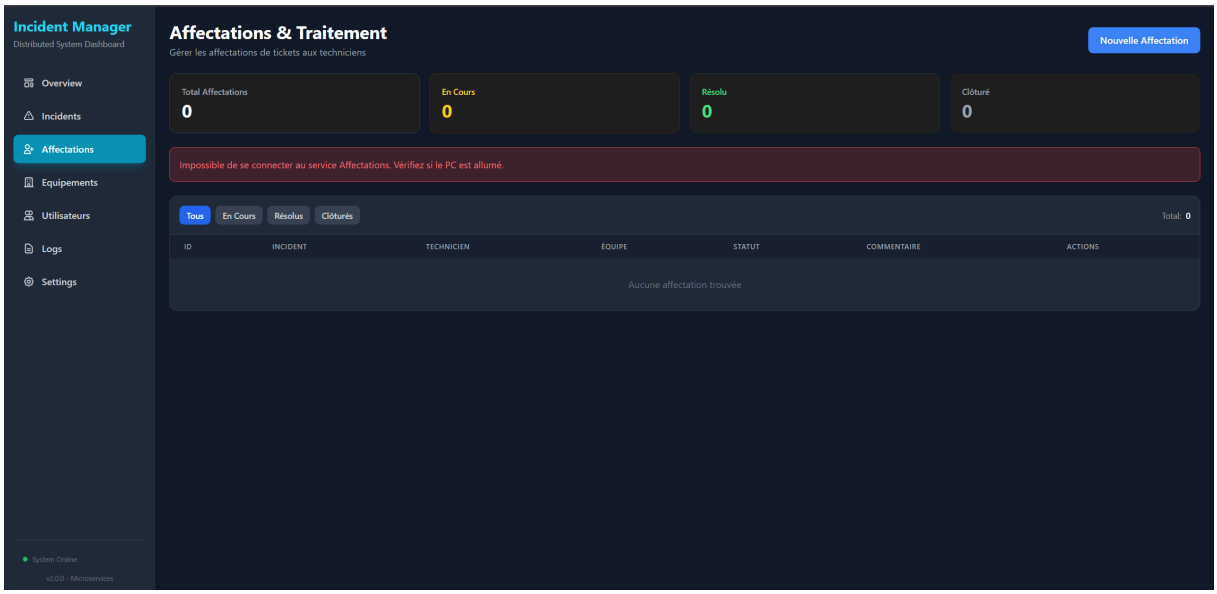

FIGURE 8.3 – Figure – Interface finale : module daffectation et de traitement

Description : cette interface permet de suivre les affectations des incidents aux techniciens, létat du traitement, ainsi que les différentes statistiques liées au cycle de résolution.

8.5 Module de gestion des équipements

Le module *Equipements* correspond au microservice Équipements et Ressources IT. Il permet dafficher le parc matériel, son état global, les catégories disponibles et les changements détat des ressources techniques.

Cette vue met en avant une présentation plus détaillée des équipements sous forme de cartes fonctionnelles avec leurs statuts respectifs.

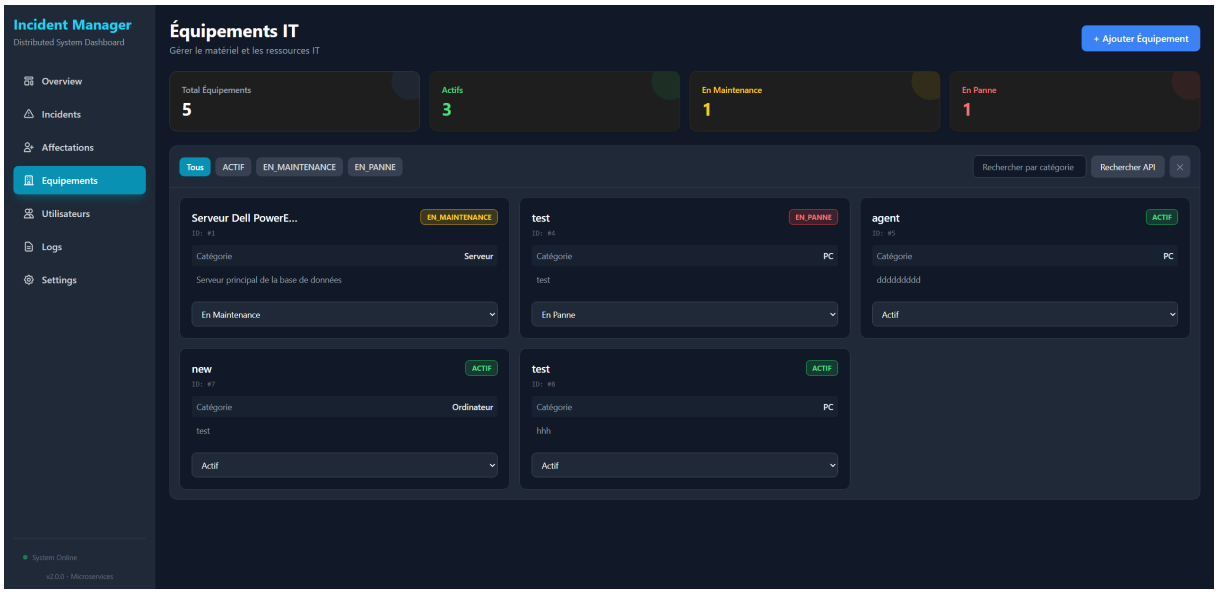

FIGURE 8.4 – Figure – Interface finale : module de gestion des équipements

Description : cette vue présente la liste des équipements informatiques, leur catégorie, leur état courant et les indicateurs de suivi du parc matériel dans le système distribué.

8.6 Module de gestion des utilisateurs

Le module *Utilisateurs* correspond à lintégration visuelle du microservice Utilisateurs et Équipes. Il doit permettre dafficher les utilisateurs, techniciens et équipes dintervention, ainsi que leurs statistiques globales.

Dans la capture fournie, le service apparaît hors ligne, ce qui met en évidence la logique de supervision et de détection de panne intégrée dans linterface finale.

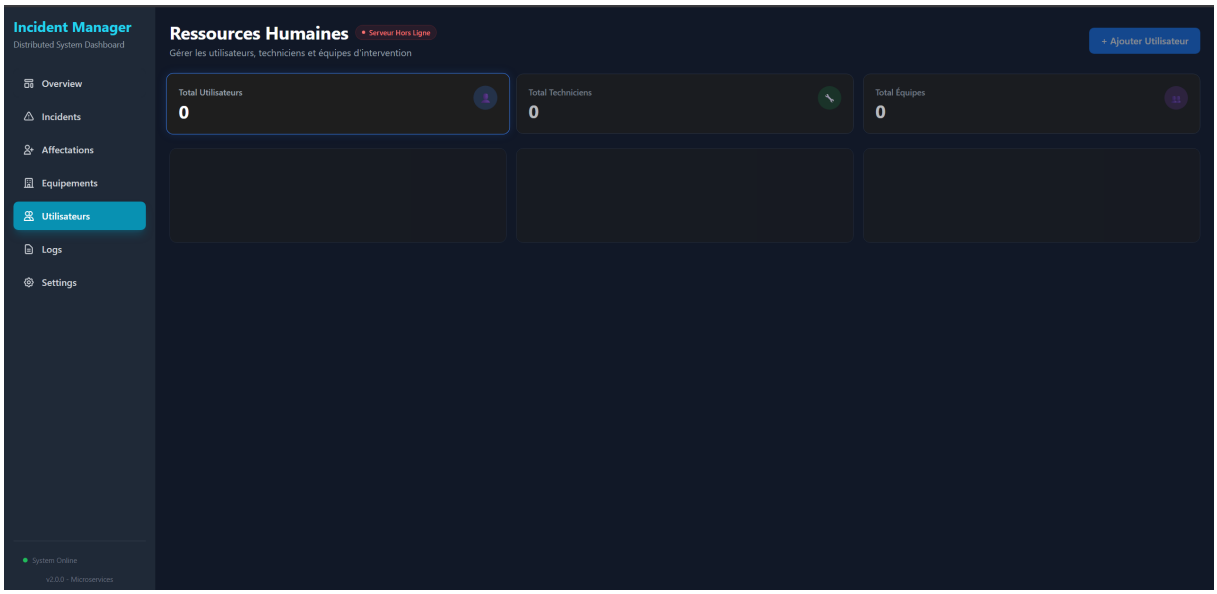

FIGURE 8.5 – Figure – Interface finale : module de gestion des utilisateurs

Description : cette interface regroupe la gestion des utilisateurs, techniciens et équipes dintervention. Elle affiche aussi létat du service afin de signaler toute indisponibilité réseau ou applicative.

8.7 Module de journalisation et dactivité

Le module *Logs* a pour objectif de centraliser lhistorique dactivité du système, les événements importants et les journaux fonctionnels issus des microservices. Il complète la dimension de traçabilité du projet par une représentation visuelle consolidée.

Cette section améliore la lisibilité des opérations importantes et facilite le suivi global de lactivité du système distribué.

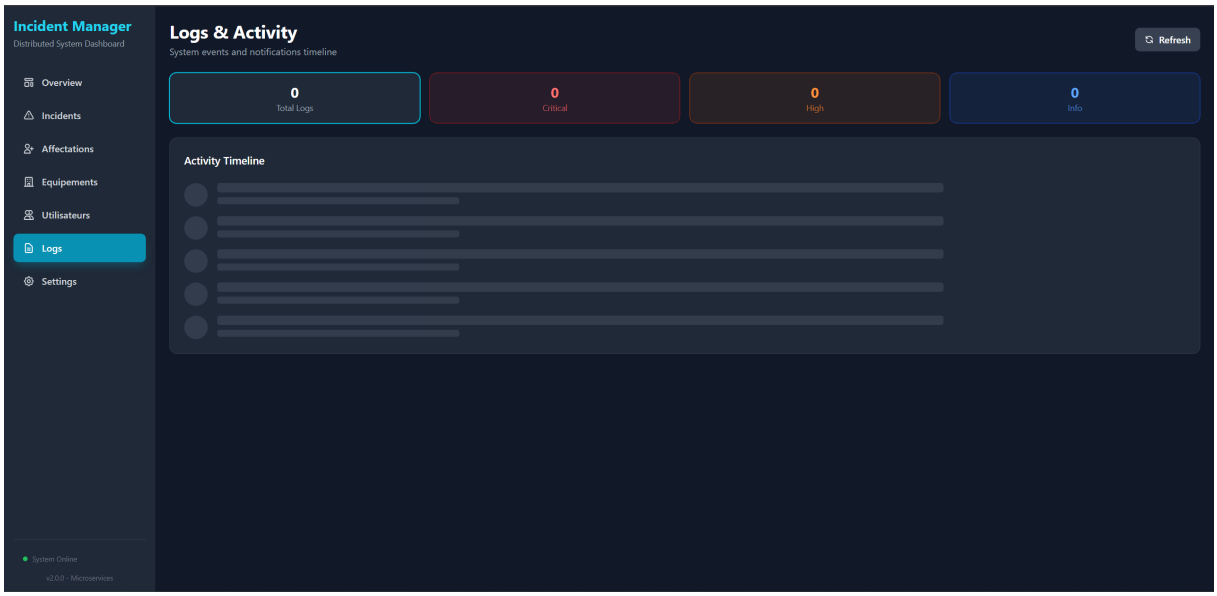

FIGURE 8.6 – Figure – Interface finale : module de logs et activité

Description : cette vue est dédiée à la visualisation de lactivité récente, des journaux système et des événements importants afin de renforcer la traçabilité globale de la plateforme.

8.8 Module de configuration centralisée

Le module *Settings* permet de configurer dynamiquement les adresses des différents microservices. Cette fonctionnalité est particulièrement importante dans un environnement distribué, car les adresses IP et les ports peuvent varier selon la machine utilisée par chaque membre du groupe.

Les URLs sont stockées localement et utilisées par linterface pour contacter les services. Cette approche facilite lintégration finale et évite de modifier le code frontend à chaque changement denvironnement.

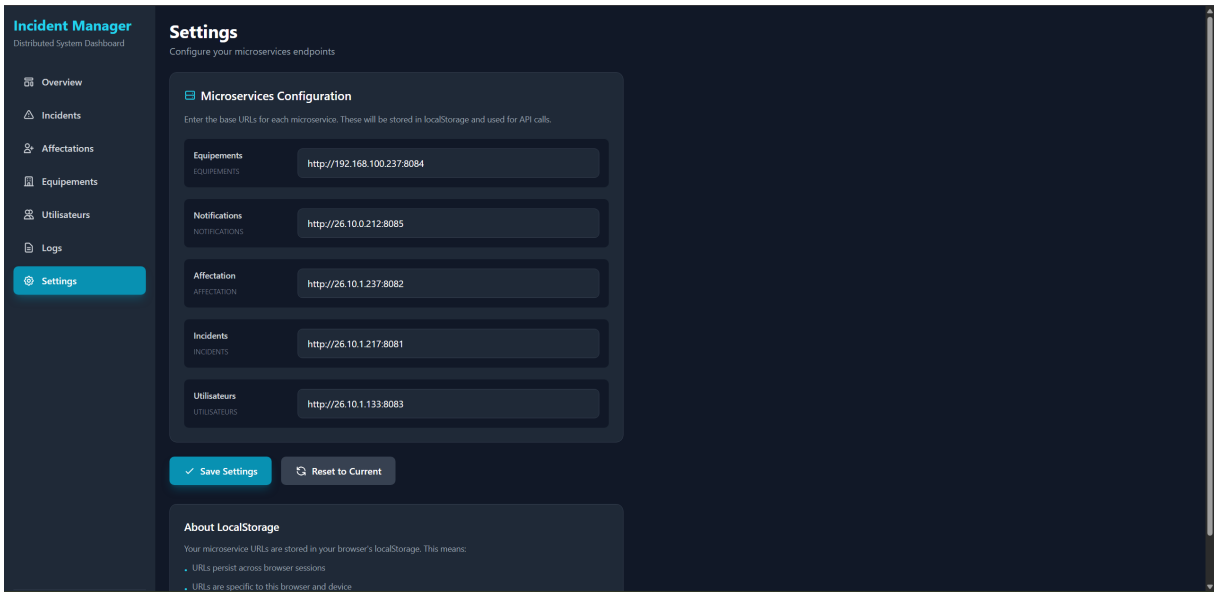

FIGURE 8.7 – Figure – Interface finale : module de configuration des microservices

Description : cette vue permet de configurer les URLs des microservices du groupe depuis linterface elle-même, ce qui simplifie les tests dintégration et ladaptation aux changements dadresses IP sur le réseau local.

8.9 Apport fonctionnel de linterface finale

La réalisation de cette interface finale apporte plusieurs avantages importants au projet :

— une vision unifiée de tous les microservices ;
— une meilleure lisibilité pour la démonstration et la soutenance ;
— une supervision simplifiée de létat des services ;
— une centralisation des accès fonctionnels ;
— une meilleure expérience utilisateur par rapport à des interfaces totalement séparées.

Elle représente donc une couche dintégration visuelle au-dessus des microservices existants, sans remettre en cause leur autonomie technique ni leurs bases de données indépendantes.

8.10 Synthèse

Linterface finale constitue laboutissement visuel du projet. Elle traduit concrètement le fonctionnement distribué du système sous une forme exploitable, claire et démontrable.

Grâce à cette centralisation, le projet ne se limite plus à une juxtaposition de microservices indépendants, mais prend la forme dune plateforme cohérente, supervisable et orientée utilisateur.

Chapitre 9

Conclusion générale

Ce projet de système distribué de gestion des incidents IT a permis de concevoir, développer et intégrer plusieurs microservices spécialisés, chacun responsable dune partie précise du cycle de traitement des incidents. Larchitecture retenue repose sur les principes fondamentaux des microservices : séparation claire des responsabilités, bases de données indépendantes, communication via API REST et possibilité dévolution modulaire.

Le microservice de déclaration constitue le point dentrée du système. Le microservice daffectation assure la prise en charge opérationnelle des incidents. Le microservice Utilisateurs et Équipes joue le rôle de service de validation des identités et des ressources humaines. Le microservice Équipements et Ressources IT fournit le référentiel matériel nécessaire à la cohérence du traitement. Enfin, le microservice Notifications et Historique assure la traçabilité transversale des événements et renforce lauditabilité du système.

Au-delà du développement individuel de chaque microservice, le projet a également nécessité une forte coordination collective : harmonisation des ports, préparation des URLs inter-services, tests dintégration, gestion du réseau local, et mise en place dune interface finale centralisée. Cette dernière a permis de donner au projet une véritable cohérence visuelle et fonctionnelle en réunissant les différents services dans une plateforme unique de supervision et daccès.

Ainsi, le travail réalisé répond aux objectifs pédagogiques et techniques du module Systèmes distribués. Il montre non seulement la maîtrise des concepts de base de Spring Boot, JPA, REST et architecture distribuée, mais aussi la capacité à intégrer plusieurs composants dans une solution globale cohérente. Ce projet constitue enfin une base solide pour de futures extensions, telles que lajout dauthentification, de reporting avancé, de monitoring temps réel ou de déploiement conteneurisé.